# Parsek Flight Recorder — Design Document

*Comprehensive design specification for Parsek's vessel recording, playback, multi-vessel tracking, and timeline system — including the vessel interaction paradox resolution, ghost chain model, relative-state time jump, and spawn safety.*

*Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones.*

**Status:** shipped; current to Parsek 0.10.5 (owner rulings through 2026-10-01). Sections 9A.1-9A.4 are retained as history of the removed chain mode, as their own note says.

---

## 1. Introduction

This document specifies the full architecture of Parsek's flight recorder system. It covers:

- The recording tree model (DAG structure) and vessel identity
- Segment boundary rule (only structural separation creates segments)
- Within-segment events for controller changes, part destruction, etc.
- Crash breakup coalescing
- Multi-vessel recording sessions with focus switching, and Switch-To / Fly continuation segments
- Segment taxonomy (environment, reference frame, rendering zones)
- Background vessel recording in the physics bubble
- Orbital checkpoint system for on-rails vessels
- Looped recording playback anchored to real vessels
- Multi-vessel merge to a single committed timeline
- Interaction with pre-existing persistent vessels and the vessel interaction paradox
- Ghost chain resolution: ghosting, spawn suppression, chain-tip spawning
- Spawn safety: collision detection, ghost extension, terrain correction, trajectory walkback
- Relative-state time jump for preserving rendezvous geometry across ghost windows
- Selective spawning UI for choosing which ghosts to materialize
- Ghost world presence: tracking station, map view, CommNet relay
- Distance-based rendering zones and performance budget
- Rewind and fast-forward with ghost playback
- Ghost implementation constraints, ghost FX and audio, mod FX recovery, and watch mode (camera follow)
- Recording file format
- Error recovery and schema compatibility

Terminal-state and synthetic-tail reliability across scene exit, vessel unload/delete, crash, and background recording end paths is specified in `parsek-recording-finalization-design.md`. That companion document supersedes older scene-exit-only assumptions whenever a recording can end before a live vessel is available for finalization.

---

## 2. Design Philosophy

These principles govern every design decision in the flight recorder. They are listed here because they inform every section that follows.

### 2.1 Recording and Playback

1. **Controller-based identity.** Vessel identity follows command authority, not KSP's unstable persistentId. A vessel is defined by its controllers, not its root part.

2. **Structural separation only.** Segment boundaries occur only when the vessel physically splits or merges. Controller changes, part destruction without separation, and crew movement within a vessel are SegmentEvents within a continuing segment. A

player who launches a vessel, partially crashes it, and recovers the wreckage sees ONE recording — not multiple disconnected segments.

3. **Highest fidelity wins.** Multiple overlapping data sources are merged by selecting the best at each UT: active > background > checkpoint.

4. **Single current recording contract.** Pre-1.0, there is no additive format evolution and no backward compatibility. Every recording carries a format version and a schema-generation discriminator (`RecordingFormatVersion = 1`, `RecordingSchemaGeneration = 4`); loads reject any mismatched format version or older/newer generation via `RecordingStore.IsRecordingSchemaCompatible`. There are no migration or forward-tolerance paths - each generation bump deletes the prior generation's tolerance seams.

## 2.2 Ghost Chains and Paradox Prevention

5. **Ghosts are the only reliable paradox prevention.** UI blocks and reservation systems can be bypassed by physics. A ghost vessel has no physics, no colliders, no docking ports — physical interaction is impossible by construction.

6. **Ghost until tip of chain.** A vessel claimed by a committed recording stays ghost until the final committed interaction in its lineage completes. The real vessel spawns only at the chain tip.

7. **Physical state is the trigger.** Only events that change a vessel's orbit, part configuration, resources, or crew force ghosting. Observation and cosmetic changes do not.

8. **Time warp is the workaround.** Players time-warp past ghost windows. In stock KSP with no time-dependent consumables, this has zero gameplay cost. The relative-state time jump (Section 14.5) eliminates the rendezvous-drift cost.

9. **First loop is real, rest are visual.** Looped recordings only spawn real vessels on the first iteration. Subsequent loops are ghost-only scenery. This prevents a looped docking recording from permanently ghosting a station.

10. **No deletion, no paradox.** Committed recordings are permanent. The ghost chain can only grow (new recordings added), never shrink (recordings removed). This guarantees monotonic convergence to a consistent timeline.

11. **Physical interactions and resource effects are decoupled.** Ghost chains control what the player can touch. The game actions timeline controls what resources exist. Neither system depends on the other. Ghosting a vessel does not affect its resource history.

12. **Paradox prevention is enforced by the physics engine, not by validation logic.** The player cannot create a conflicting recording because conflicting interactions are physically impossible — ghost vessels have no colliders, no docking ports, no physics presence. This is not a rule that can be bypassed by mods, exploits, or the debug menu. It is a property of the simulation itself.

## 2.3 Time Jump

The time jump is a specialized mechanism for **multi-vessel rendezvous configurations** where the player has set up an approach to a ghosted vessel and needs to skip past its chain tip without losing the alignment. Normal time warp (Section 14.4) remains the default way to advance time — the timeline plays out, ghosts appear and despawn, vessels spawn at their endpoints. The time jump exists only because normal warp destroys carefully aligned approach geometry through differential Keplerian drift.

13. **Nothing moves, the clock jumps.** During a time jump, the entire physics bubble stays frozen in place. Only UT changes. Orbital epochs are adjusted to maintain Keplerian consistency. This eliminates drift by construction.

14. **Selective spawning, chronological constraint.** The player picks which chain tip to jump to. Earlier independent tips are auto-spawned (ghosts cannot exist past their tip). Later tips stay ghost.

15. **Time jump complements warp, does not replace it.** Fast-forward via normal time warp is always available and plays out the timeline naturally. The time jump is an additional tool specifically for preserving bubble geometry when the player needs to interact with a vessel that is currently ghost.

16. **Visual honesty.** Planet rotation, sun angle, and star field change to reflect the new UT. The player sees that time passed. Only the local bubble geometry is preserved.

17. **No persistent side effects.** The time jump is equivalent to a warp — it advances UT and processes the timeline. Rewind undoes it normally. Recordings are not modified.

18. **Game actions processed atomically.** All resource recalculation happens once at the target UT, same as warp exit.

### 2.4 Ghost World Presence

19. **Ghosts participate selectively.** Ghosts are lightweight (raw GameObjects), not full KSP Vessels. They participate in CommNet and map view through selective API registration. They do not participate in any active game system. This preserves the paradox-proof boundary while allowing the game world to feel correct.

---

## 3. Vessel Identity Model

### 3.1 Why KSP's Vessel ID Is Insufficient

KSP assigns each vessel a `persistentId`. This ID is unstable across vessel lifecycle events: when a vessel decouples, one side inherits the original ID (the side with the root part) and the other gets a new ID. When two vessels dock, one ID survives and the other is destroyed. The surviving ID depends on which vessel was active at docking time.

This means a vessel that launches as one craft, drops stages, undocks a lander, docks to a station, and later gets recycled will have its KSP vessel ID change multiple times in ways that don't correspond to what the player considers "the same thing."

### 3.2 Controller-Based Identity

Parsek defines vessel identity around **command authority** — the presence of a controller part that gives a vessel agency.

**Controller parts** (in priority order):

- Crewed command pod (Mk1, Mk1-3, etc.)
- Occupied external command seat
- Probe core (with or without electric charge)
- Kerbal on EVA

**A vessel segment** is the continuous history of a command-capable assembly of parts, from the moment it gains command authority to the moment it loses it permanently.

**Gains command authority:** A controller part becomes the root of an independent vessel — either at launch, at a split event where this side has a controller, or via construction (a kerbal places and activates a probe core).

**Loses command authority permanently:** Recovery, destruction, recycling, or removal of all controller parts with no replacement.

### 3.3 Debris

An assembly of parts with no controller is debris. Debris that separates from a recorded vessel gets a short recording of its own (`Recording.IsDebris = true`, parent-anchored through `ParentAnchorRecordingId`) so the ghost shows spent stages falling away. That recording ends after `BackgroundRecorder.DebrisTTLSeconds` (60 s), on destruction, or when the debris leaves the physics bubble, whichever comes first. Parsek does not track debris beyond that window; KSP's own persistence handles debris positions afterwards.

**A crewed part with no controller is not debris** (todo SPLIT-OF-A-CREWED-COMMANDLESS-PART-READS-AS-DEBRIS, 2026-10-22). A separated assembly that carries a kerbal but no command module (a Hitchhiker or a Mk1 crew cabin decoupled or undocked with someone inside) is recorded as its own branch like a controlled vessel: no 60 s lifetime, it records until it ends and spawns at its end like any other vessel, so the kerbal inside is never lost with a debris recording. It is still not a controllable vessel: it is never a Re-Fly slot and does not count toward a Rewind Point (`ParsekFlight.ShouldRecordAsOwnBranch` decides recording, `IsTrackableVessel` stays the controllability test).

If a controlled vessel acquires debris (e.g., claw grab of a fuel tank), that's a merge event where one parent is a controller-based vessel and the other is inert. The recording notes the acquisition; the debris's prior existence is irrelevant to the recording.

If a vessel loses all controllers (crew removed, probe core out of EC with no restoration), its active segment ends and it becomes debris. If a controller is later restored (another vessel docks and provides control, a kerbal boards it, EC is restored to a probe core), a new active segment begins.

---

# 4. The Recording Tree (DAG)

## 4.1 Structure

A recording tree is a **directed acyclic graph (DAG)** of vessel segments connected by lifecycle events. It is not a simple tree because merge events (docking, boarding) join branches back together.

## 4.2 Terminology

This document uses two layers of naming. The **design model** uses conceptual names (VesselSegment, TreeEvent) to describe what the DAG represents. The **implementation** uses concrete class names (Recording, BranchPoint) that appear in code. The mapping:

| Design concept | Implementation class | Meaning |
|---|---|---|
| VesselSegment | `Recording` | One vessel's continuous trajectory + events |
| TreeEvent | `BranchPoint` | A DAG node where vessels split, merge, launch, or terminate |
| RecordingTree | `RecordingTree` | The complete DAG for one mission |

Throughout this document, "Recording" (capitalized) refers to the per-vessel data object. "Committed tree" or "mission recording" refers to a RecordingTree that has been committed to the timeline. The word "recording" (lowercase) refers to the act of recording.

## 4.3 Authoritative Data Model

These are the actual data structures. All types referenced elsewhere in this document are defined here.

**RecordingTree** — the complete DAG for one mission. Committed trees are stored in a flat list (`RecordingStore.CommittedTrees`) that survives scene changes via static field persistence and is serialized/deserialized by the ScenarioModule (`ParsekScenario`). The ghost chain walker iterates this list on every rewind and save load.

```
RecordingTree
  Id:                 string — unique identifier
  TreeName:           string
  RootRecordingId:    string — the first Recording (launch)
  ActiveRecordingId:  string — currently focused Recording (changes on focus switch)
  Recordings:         dict of {recordingId: Recording} — all vessel recordings in this tree
  BranchPoints:       list of BranchPoint — DAG events connecting Recordings
  BackgroundMap:      dict of {vesselPid: recordingId} — background vessel mappings (runtime only)
  PreTreeFunds:       double — game state at tree start (for delta computation)
  PreTreeScience:     double
  PreTreeReputation:  float
  DeltaFunds:         double — net resource change from this tree
  DeltaScience:       double
  DeltaReputation:    float
  ResourcesApplied:   bool
```

**Recording** — one vessel's continuous trajectory, events, and state. This is the VesselSegment of the design model. A Recording covers the lifetime of one command-capable assembly from creation to termination (or ongoing).

```
Recording
  RecordingId:              string — unique identifier
  TreeId:                   string — which RecordingTree this belongs to
  VesselName:               string
  VesselPersistentId:       uint — KSP's vessel PID at recording time
  RecordingFormatVersion:   int - recording/sidecar format version (currently 1)
  RecordingSchemaGeneration: int - clean-slate schema discriminator (currently 4); load rejects a mismatched format ve
rsion or an older/newer generation

  // Trajectory data (primary)
  TrackSections:            list of TrackSection — typed trajectory chunks (Section 6)
  // Trajectory data (flat legacy representation, dual-written alongside TrackSections)
  Points:                   list of TrajectoryPoint — flat trajectory
  OrbitSegments:            list of OrbitSegment — flat orbit segments

  // Events
  PartEvents:               list of PartEvent - discrete visual state changes (45 event types, `PartEventType`)
  SegmentEvents:            list of SegmentEvent — within-segment state changes (Section 4.8)
```

```
  Controllers:              list of ControllerInfo — controller parts at segment start
  IsDebris:                 bool — true if no controllers

  // DAG linkage
  ParentBranchPointId:      string — BranchPoint that created this Recording
  ChildBranchPointId:       string — BranchPoint that ended this Recording
  ParentRecordingId:        string — parent in the tree

  // Terminal state (how/where the vessel ended)
  TerminalStateValue:       RECOVERED | DESTROYED | RECYCLED | DESPAWNED | null
  TerminalOrbit*:           Keplerian elements at segment end (inc, ecc, sma, lan, argPe, mAe, epoch, body)
  TerminalPosition:         SurfacePosition — lat/lon/alt if surface-landed at end
  TerrainHeightAtEnd:       double — terrain height at final position (NaN if not surface; for spawn correction)

  // Vessel snapshots (ConfigNode from KSP's vessel.BackupVessel() — complete vessel state)
  VesselSnapshot:           ConfigNode — full vessel state: part tree, resources, crew, orbit, action groups
  GhostVisualSnapshot:      ConfigNode — visual-only snapshot for ghost mesh building

  // Loop configuration
  LoopPlayback:             bool
  LoopIntervalSeconds:      double
  LoopAnchorVesselId:       uint — anchor vessel for relative loop positioning
  LoopAnchorBodyName:       string — expected body (for drift validation)

  // Spawn tracking
  VesselSpawned:            bool — has this Recording's vessel been spawned into the game?
  SpawnedVesselPersistentId: uint — PID of the spawned vessel (0 = not spawned)

  // No stored antenna data: the ghost CommNet node derives it at load from
  // VesselSnapshot plus part prefabs (section 15.6).
```

**VesselSnapshot explained:** A snapshot is a KSP `ConfigNode` produced by `vessel.BackupVessel()`. It contains the complete serialized vessel state: the entire part tree with every module's persistent data, crew assignments, resource levels, orbital elements, vessel situation, action group states, and discovery info. Snapshots are captured at two points: (1) at recording commit time (stored on the Recording), and (2) at ghost conversion time (captured from the live vessel before despawn). For chain-tip spawning, the snapshot from the tip Recording is used — for a chain bare-S → S+A → S+A+B, the S+A+B snapshot comes from R2's Recording of the merged vessel at its endpoint.

**Finalization note:** `TerminalStateValue`, `ExplicitEndUT`, terminal orbit/surface metadata, and predicted `OrbitSegment` tails must be finalized consistently even if KSP destroys or unloads the vessel before scene exit. The reliability contract and cache design are documented in `parsek-recording-finalization-design.md`.

**BranchPoint** — a DAG node where the tree branches or merges. This is the TreeEvent of the design model.

```
BranchPoint
  Id:                       string — unique identifier
  UT:                       double — universal time
  Type:                     BranchPointType enum:
                              Undock=0, EVA=1, Dock=2, Board=3, JointBreak=4,
                              Launch=5, Breakup=6, Terminal=7,
                              VesselSwitchContinuation=8, GroundPartPlaced=9 (Section 4.11)
  ParentRecordingIds:       list of string — Recordings that ended here
  ChildRecordingIds:        list of string — Recordings that began here

  // Type-specific metadata (nullable, set based on Type)
  SplitCause:               DECOUPLE | UNDOCK | EVA
  DecouplerPartId:          uint — part that triggered separation
  BreakupCause:             CRASH | OVERHEAT | STRUCTURAL_FAILURE
  BreakupDuration:          double — time window of the breakup
  DebrisCount:              int — non-tracked fragment count
  CoalesceWindow:           double — threshold used (default 0.5s)
  MergeCause:               DOCK | BOARD | CONSTRUCT | CLAW
  TargetVesselPersistentId: uint — pre-existing vessel's PID (links to game state; used by chain walker)
  TargetVesselGuid:         string - that vessel's launch guid (Vessel.id) at couple time; optional, null when unknown
  TerminalCause:            RECOVERED | DESTROYED | RECYCLED | DESPAWNED
```

`VesselSwitchContinuation` is not a physical event: it attaches the recording that starts when the player takes a vessel over through a stock Fly or Switch-To button (Section 5.4) under that vessel's last recording in the tree, so its history stays one lineage across the gap. When no earlier recording of the vessel exists, the new recording is a tree root and no branch point is written.

**TrajectoryPoint** — position sample. Recorded at up to 50 Hz in atmospheric flight, down to 0.1 Hz for surface-stationary. The most-sampled data structure in the system.

```
TrajectoryPoint
  ut:         double — universal time
  latitude:   double — body-relative
  longitude:  double — body-relative
  altitude:   double — above sea level
  rotation:   Quaternion — vessel attitude
  velocity:   Vector3 — vessel velocity
  bodyName:   string — reference body
  funds:      double — game state at this moment (for resource delta tracking)
  science:    float
  reputation: float
```

**OrbitSegment** — Keplerian orbital elements for on-rails coasting. Used within ORBITAL_CHECKPOINT TrackSections and as the legacy flat orbit list.

```
OrbitSegment
  startUT:                   double
  endUT:                     double
  inclination:               double
  eccentricity:              double
  semiMajorAxis:             double
  longitudeOfAscendingNode:  double
  argumentOfPeriapsis:       double
  meanAnomalyAtEpoch:        double
  epoch:                     double
  bodyName:                  string
  orbitalFrameRotation:      Quaternion
  angularVelocity:           Vector3
```

**TrackSection** — a typed trajectory chunk with a single environment + reference frame combination.

```
TrackSection
  environment:               SegmentEnvironment (ATMOSPHERIC | EXO_PROPULSIVE | EXO_BALLISTIC |
                               SURFACE_MOBILE | SURFACE_STATIONARY | APPROACH)
  referenceFrame:            ReferenceFrame (ABSOLUTE | RELATIVE | ORBITAL_CHECKPOINT)
  source:                    TrackSectionSource (ACTIVE | BACKGROUND | CHECKPOINT)
  startUT:                   double
  endUT:                     double
  anchorVesselId:            uint - RELATIVE frame: live-PID anchor (used by loop playback)
  anchorRecordingId:         string - RELATIVE frame: recorded-anchor trajectory (used by non-loop playback)
  frames:                    list of TrajectoryPoint - ABSOLUTE frames, or RELATIVE anchor-local metre offsets
  bodyFixedFrames:           list of TrajectoryPoint - RELATIVE only: body-fixed world-coordinate primary playback su
rface for parent-anchored recordings
  checkpoints:               list of OrbitSegment — for ORBITAL_CHECKPOINT
  sampleRateHz:              float — actual recording sample rate
  boundaryDiscontinuityMeters: float — position gap vs previous section end
```

**PartEvent** - a discrete visual state change on a specific part. 45 event types (`PartEventType` in `PartEvent.cs`; values from 34 on are explicitly numbered and append-only) covering engines, parachutes, solar panels, antennas, lights, landing gear, cargo bays, fairings, RCS, robotics, thermal animations, inventory deployables, resource converters, and EVA jetpack / ragdoll state.

```
PartEvent
  ut:              double
  partPersistentId: uint — which part
  eventType:       PartEventType (EngineIgnited, EngineShutdown, EngineThrottle,
                     DeployableExtended, DeployableRetracted, LightOn, LightOff,
                     GearDeployed, GearRetracted, ParachuteDeployed, ParachuteCut,
                     CargoBayOpened, CargoBayClosed, FairingJettisoned, RCSActivated,
                     RCSStopped, Decoupled, Destroyed, Docked, Undocked, ... 45 total)
  partName:        string
  value:           float — event-specific (e.g., throttle percentage)
  moduleIndex:     int — which module on the part (for multi-module parts)
```

**SegmentEvent** — see Section 4.8.

**ControllerInfo** — descriptor for a controller part.

```
ControllerInfo
  type:            ControllerType (CrewedPod | ExternalSeat | ProbeCore | EVAKerbal)
  partName:        string
  partPersistentId: uint
```

**GhostAntennaSpec** ( `GhostCommNet.cs` ) - in-memory antenna data for a ghost CommNet node, derived from the vessel snapshot and the part prefabs and never serialized (KSP does not persist antenna power in a snapshot). See section 15.6.

## 4.4 The Segment Boundary Rule

**CRITICAL: Only physical structural separation creates new segments. Controller changes never create segment boundaries.**

A segment boundary (TreeEvent) occurs ONLY when:

- The vessel physically splits into two or more independent assemblies (SPLIT, BREAKUP)
- The vessel physically merges with another assembly (MERGE)
- The vessel ceases to exist (TERMINAL)
- The vessel is created (LAUNCH)

A segment boundary does NOT occur when:

- A controller part is destroyed but the vessel remains one connected assembly
- A controller part loses power, hibernates, or becomes non-functional
- A controller part regains power or is repaired
- The player uses "Control From Here" to change the active control point
- KSP internally reassigns the root part
- Crew transfers between parts within the same vessel
- Parts are destroyed, added, or removed without splitting the vessel

These non-structural changes are recorded as SegmentEvents (Section 4.8) within the continuing segment.

## 4.5 Identity Persistence Rule

A vessel segment persists as long as at least one controller part **physically exists** on the vessel, regardless of whether that controller is currently **functional**.

- A probe core with no electric charge is still a controller (it physically exists).
- A command pod with no crew is still a controller (the part exists).
- A hibernating probe core is still a controller.
- A damaged but not destroyed controller is still a controller.

Only physical destruction or removal of ALL controller parts ends the segment.

## 4.6 Split Events

When a vessel physically separates into independent assemblies (staging, undocking, EVA), the parent segment ends and two or more child segments begin. Each child inherits whatever controller parts end up on its side. Children without any controller are tagged as debris and receive minimal recording (position only until destruction/despawn).

An EVA is a split in the recording DAG like the others, but it is NOT a rewind split: Re-Fly is for vessel separations only (staging, decoupling, undocking; owner ruling 2026-09-27), so an EVA BranchPoint never carries a Rewind Point and an EVA kerbal is never an Unfinished Flight. See `parsek-rewind-to-separation-design.md` section 1.5.

## 4.7 Breakup Events (Crash Coalescing)

Crashes, overheating, and structural failures can produce many fragments across multiple physics frames. Rather than recording 20 individual SPLIT events in rapid succession, Parsek coalesces all separation events within a short time window (default 0.5 seconds) into a single BREAKUP tree event.

The BREAKUP event records the parent segment, all child segments that have controllers (these get proper segments with trajectory recording), and a count of debris fragments (not individually tracked).

**Example — Mun crash with partial survival:**

```
Vessel with probe core A (front) and probe core B (rear) crashes.
Multiple parts break off across 0.3 seconds.

Parsek records:
  BREAKUP at T+0.00 (duration: 0.30s)
    parent: [original vessel]
    controlledChildren:
      - [assembly containing probe core A] -> new segment
      - [assembly containing probe core B] -> new segment
    debrisCount: 14
```

### 4.8 Within-Segment Events (SegmentEvents)

SegmentEvents record significant state changes that do NOT create DAG branches. They are annotations on a continuing segment, important for crew tracking, ghost visual fidelity, and vessel state history.

```
SegmentEvent
  type:      CONTROLLER_CHANGE | CONTROLLER_DISABLED | CONTROLLER_ENABLED |
             CREW_LOST | CREW_TRANSFER | PART_DESTROYED | PART_REMOVED | PART_ADDED |
             TIME_JUMP
  ut:        universal time
  details:   type-specific data
```

Types:

- **CONTROLLER_CHANGE** — A controller part is destroyed, added, or changes type. Lists lost controllers, gained controllers, and remaining controllers. The segment continues.
- **CONTROLLER_DISABLED** — Controller exists but is non-functional (no EC, hibernation, damaged). Segment continues. Identity persists.
- **CONTROLLER_ENABLED** — Controller regains functionality (EC restored, repaired, wake from hibernation).
- **CREW_LOST** — Crew member died (destroyed or MIA). Relevant for crew tracking.
- **CREW_TRANSFER** — Crew moved between parts within the same vessel (not via EVA — EVA is a SPLIT/MERGE in the DAG). Records kerbal name, from-part, to-part.
- **PART_DESTROYED** — Part destroyed without splitting the vessel. Ghost stops rendering this part mesh after this UT. Records part name, part ID, whether it was a controller.
- **PART_REMOVED** — Part removed via construction without splitting the vessel. Also a stored part taken out of a foreign vessel's inventory (Section 4.12).
- **PART_ADDED** — Part added via construction without creating a new vessel. Ghost starts rendering this part mesh after this UT. Also a stored part put into a foreign vessel's inventory (Section 4.12).
- **TIME_JUMP** — Discontinuity from a relative-state time jump (see Section 14.5). Stores pre-jump and post-jump state vectors (position, velocity, UT) so playback can handle the gap as a visual cut.

### 4.9 Merge Events

When two vessels merge (docking, boarding, claw attachment), both parent segments end and one child segment begins. The child inherits all controllers from both parents.

**Merging with a pre-existing persistent vessel** (e.g., docking to a station from a previous recording): The current recording's segment ends with a merge event. The `targetVesselId` field links the recording tree to the persistent game state. The combined vessel is a new segment in the current recording's tree. The historical recording tree of the station is NOT modified — it is immutable.

### 4.10 Example DAG

A crewed lander with a probe-controlled transfer stage:

```
LAUNCH (capsule + transfer + booster)
  |
  +- SPLIT: staging
  |    +- [booster] -- no controller -- DEBRIS
  |    +- [capsule + transfer] -- controller: capsule crew
  |         |
  |         +- SPLIT: undock
  |         |    +- [capsule/lander] -- controller: crew
  |         |    |     +- SPLIT: EVA
  |         |    |     |    +- [kerbal] -- controller: self
  |         |    |     |    |     +- MERGE: board
  |         |    |     |    +- [lander] -- controller: remaining crew
  |         |    |     |         +- MERGE: board
  |         |    |     +- [reunified lander]
  |         |    |          +- MERGE: dock with transfer
  |         |    +- [transfer] -- controller: probe core
  |         |         +- MERGE: dock (lander returns)
  |         |
  |         +- [reunified vessel]
  |              +- SPLIT: decouple for reentry
  |              |    +- [capsule] -- RECOVERED
  |              |    +- [transfer] -- DESTROYED
```

### 4.11 Placement Events (EVA ground parts)

Owner ruling 2026-09-26 (todo EVA-GROUND-SCIENCE-PLACED-PART-PID-NOT-ON-VESSEL, option 1): a ground part a kerbal places on EVA (Breaking Ground experiments, power and comms units, the Central Station) gets its OWN recording and ghost, as a member of the flight's tree created at the moment of placement.

**What stock does (decompiled KSP 1.12.5).** The confirm press in `ModuleInventoryPart.OnUpdate` (gated on `vessel.isActiveVessel`, so only the active EVA kerbal can place) calls `DeployGroundPart` → `GetProtoVesselNode` → `Game.AddVessel`, which loads the ProtoVessel and fires `onNewVesselCreated(vessel)` synchronously; back in `OnUpdate`, stock then fires `onDeployGroundPart(partName)`. The new vessel is single-part and `LANDED`, and a frame or two later its `ModuleGroundSciencePart.OnStart` fires `onGroundSciencePartDeployed`. The pick-up is the part's own `RetrievePart` event: it clears `deployedOnGround`, stores the part in the kerbal's inventory, fires `onGroundSciencePartRemoved`, plays the retract animation, and `OnRetractCompleted` fires `onGroundSciencePartRemoved` a SECOND time and calls `vessel.Die()`.

**What counts as a placement.** Only `onDeployGroundPart`, matched to the vessel `onNewVesselCreated` announced in the SAME frame (single part, same part name, not already a tree member), while the active vessel is the EVA kerbal the tree is recording in the foreground. `onGroundSciencePartDeployed` is not a placement signal: it also fires whenever an unlinked experiment merely starts landed (a load into physics range, a Central Station removal), so nothing is recorded from it any more (sibling todo INVENTORY-PLACED-FIRES-ON-ANY-LANDED-EXPERIMENT-LOAD). Pure gate: `GroundPartPlacement.EvaluatePlacement`.

**Tree shape.** A new `BranchPointType.GroundPartPlaced = 9` (a purely additive enum member: no key, field or layout changes, so no schema generation bump; an older build reads it through the codec's existing unknown-type fallback). The branch point has `ParentRecordingIds = [the kerbal's recording]` and `ChildRecordingIds = [the placed part's recording]`. The kerbal's recording does NOT end and its `ChildBranchPointId` is NOT set: placing a part does not change the kerbal, the single-slot pointer stays free for the kerbal's own later split or merge (the board), and the kerbal stays an ordinary leaf instead of relying on the breakup-continuous "effective leaf" rule. The spawn decision's #114 safety net (`GhostPlaybackLogic.IsNonLeafInTree`, which reads "parent of any branch point" as "branched into a continuation") skips `GroundPartPlaced` branch points for the same reason, so a kerbal left standing on EVA after placing a part still spawns at the end of his flight (after a rewind, at the Space Center). The child is reached through its own `ParentBranchPointId`, the way foreground debris children are once their parent splits again; a walk that must own the placing recording's products finds it by parent id (the switch-segment subtree behind scoped Discard and the no-op auto-discard, `RecordingStore.CollectSwitchSegmentSubtreeRecordingIds`, so discarding a segment discards what its kerbal placed, and a placement makes the segment meaningful). The type is non-claiming for ghost chains (the kerbal's vessel is unchanged), is not a structural mutation for Re-Fly auto-seal, and reads "Placed" in the Missions composition.

**What is recorded.** The member is an ordinary background-recorded tree recording: `VesselPersistentId` = the placed vessel's pid, `RecordedVesselGuid` = its `Vessel.id` (fresh per placement, so the pid-vs-guid identity rules hold), `ExplicitStartUT` = the placement UT, `Generation` = parent + 1, `IsDebris = false` (it records until it ends, no debris TTL) and NO parent-anchor contract (it is static on the ground; anchoring it to a walking kerbal would record meaningless RELATIVE offsets). Its `VesselSnapshot` / `GhostVisualSnapshot` are captured from the live vessel one frame after the placement (in the placement frame the parts exist but have not started), and it joins the `BackgroundMap`, so the background recorder samples its static pose and refreshes its end state like any other member. Two part events go on the MEMBER, keyed by the placed part's own pid (which IS in the member's snapshot, so INV4 resolves by construction): `InventoryPartPlaced` at the placement UT and `InventoryPartRemoved` at the FIRST pick-up

signal (the second `onGroundSciencePartRemoved`, from `OnRetractCompleted`, is deduplicated). Nothing is recorded on the kerbal's recording.

**How the pick-up ends the member.** The first `onGroundSciencePartRemoved` for a member marks it retrieval-pending; when `vessel.Die()` reaches `OnVesselWillDestroy`, the existing disassembly seam (`TryStampDisassembledTerminal`) stamps `TerminalState.Disassembled` (reason `GroundPartRetrieved`) instead of the crash path's `Destroyed`. The Central Station has no `ModuleGroundSciencePart` and fires no Removed event, so the seam also accepts a live `ModuleGroundPart.deployedOnGround == false` read (stock clears it at the start of `RetrievePart`; a crash leaves it true). Disassembled is already non-spawnable and never a recovery (no funds, science or reputation), and the part really is back in an inventory.

**Spawn at end.** Nothing new: the member is a leaf. If it is still placed when the tree ends, it finalizes `Landed` with a snapshot and is a spawnable leaf, so after a rewind it comes back as a real vessel at its recorded end pose like any other vessel the tree leaves behind (the KSC exclusion-zone retirement applies to it like to any vessel: a part left on the pad is retired, not spawned). A picked-up member ends `Disassembled` and is never spawned. Members of one cluster stand a few metres apart, well inside the spawn-collision box (12.6 m for one part), so the spawn collision check and the Space Center landed de-overlap ignore a vessel Parsek spawned from another member of the same committed tree that still stands where its spawn put it (`CoexistingTreeSiblingSpawn`; the same rule lets the kerbal and the rover that end among the parts spawn there). Without it only the first member of a cluster spawns. At a member's recorded end spot the rule's time gate (recorded existence overlaps, persisting terminals running past their trimmed end) always passes for two spawnable leaves; at a walkback candidate it is real: the other member counts only if it already stood at its spot at the candidate's recorded UT, so a vehicle stepping back along its own path is never placed inside a part set down there after it drove through. The other member may be its spawn or its recorded vessel still live (a positive pid + launch Guid match, never the spawning member's own launch), measured from that vessel's latest recorded segment.

**Beside a vessel of an earlier flight** (operator ruling 2026-09-29). An EVA kerbal or a placed part is not pushed off its recorded spot by a vessel that was already standing there when its recording was made and has not moved since (the capsule the kerbal climbed out of, recorded in an earlier committed tree); a vessel that arrived later still pushes it. Parsek records no foreign vessel's position during a recording, so the proof is the blocker's own committed history (`CoexistingTreeSiblingSpawn.ClassifyOtherTree`): the blocker is positively that history's vessel (spawn stamp or pid + launch Guid, from the effective recording set), its tree was committed before the spawning member's tree, its latest segment is a standing leaf that ended at or before the spawning member arrived at its spot (a placed part's placement, an EVA kerbal's start of its final stand; a walkback candidate's UT on a walkback; a history still running then was a replay, not a vessel), and it stands within 3 m of that leaf's spot. Every collision site above and the Space Center de-overlap apply it. A vehicle spawning beside such a vessel is still nudged clear (the duplicate-delivery guard), and a vessel with no committed history, or only a later one, still pushes.

**The ghost.** The member's ghost is the placed part standing where it was placed: visible from the placement UT (the Placed event), hidden at the pick-up (the Removed event), and gone when the recording ends at the vessel's `Die()` about 4.7 s later.

**Re-Fly.** A kerbal on EVA is never re-flown (Re-Fly is for vessel separations only, owner ruling 2026-09-27), so no Re-Fly session has the placing kerbal as its root. Re-flying the vessel the kerbal came from does not reach the member either: the supersede closure (`EffectiveState.ComputeSubtreeClosureInternal`) skips the kerbal's own recording as a side-off branch (different pid) at the EVA branch point, so the kerbal and every part it placed keep their recordings, and `EffectiveState.EnqueueDebrisChildren` admits debris only. Todo REFLY-CLOSURE-OMITS-PLACED-GROUND-PARTS closed as moot on that evidence.

## 4.12 Inventory Transfers with a Foreign Vessel

Todo EVA-INVENTORY-MOVE-TO-FOREIGN-VESSEL-UNCLAIMED (2026-10-15). A kerbal on EVA can move a stored part (KSP 1.12 cargo) into or out of an inventory container on a vessel that is not part of his flight's tree: a rover parked by an earlier mission, a base. That changes the other vessel's physical state, so by Section 12.6 the flight claims it: after a rewind it must be a ghost until the flight ends and then come back in its post-transfer form. Without a record of the transfer the rewound game kept the container's old contents while the kerbal also spawned with the part (duplicated), or the part he put away vanished (deleted).

**What stock does (decompiled KSP 1.12.5).** `ModuleInventoryPart.StoreCargoPartAtSlot`, `ClearPartAtSlot`, `UpdateStackAmountAtSlot` and `RemoveNPartsFromInventory` fire `onModuleInventorySlotChanged(module, slot)` after the slot changed, on the module whose slot changed. A drag from a container into the kerbal's inventory clears the container's slot first and then fills the kerbal's (two events, one per module). `OnLoad` fires only `onModuleInventoryChanged`, never the slot event, so loading a vessel records nothing.

**What counts as a transfer.** `ForeignInventoryTransfer.Evaluate`: the tree is recording, the active vessel is the EVA kerbal whose recording is the tree's active recording, the changed module sits on another real vessel (not a ghost map vessel, not an EVA kerbal) and the change is not one of Parsek's own supply-route writes (`ParsekInventoryWrites`, set around the pickup and delivery writers). A vessel no recording of the tree carries is adopted; a vessel adopted by an earlier transfer gets one more event; any other tree vessel (the kerbal's own lander) is left to its own recording, whose end snapshot already carries the change.

**Tree shape.** A new `BranchPointType.InventoryTransfer = 10` (purely additive: no schema generation bump), the same shape as `GroundPartPlaced`: `ParentRecordingIds = [the kerbal's recording]`, `ChildRecordingIds = [the foreign vessel's recording]`, and the kerbal's `ChildBranchPointId` is not set, so he keeps recording and stays a leaf. Every walk that treats a placement as the kerbal's product reads both types through one predicate, `RecordingTree.IsKerbalSideBranch`: the spawn decision's #114 safety net, the Re-Fly downstream check, the session-suppressed and supersede closure, and (generically, by parent id) the switch-segment subtree behind scoped Discard. The branch point is non-claiming and names no target vessel; it reads "Cargo" in the Missions composition.

**What is recorded.** The member is an ordinary background-recorded recording of the foreign vessel: its pid, its `Vessel.id` as `RecordedVesselGuid`, `ExplicitStartUT` = the transfer UT, `Generation` = parent + 1, not debris, no parent anchor. Its snapshot is taken at the transfer (after the change) and refreshed at every later transfer, so the end snapshot is the post-transfer vessel; it joins the `BackgroundMap`. Each transfer is one `SegmentEvent` on the member, `PartRemoved` or `PartAdded` (fewer or more parts in the slot than the last reading of it, else the vessel's saved `ProtoPartSnapshot`; with neither, an emptied slot reads as a removal), with details `inventory part=<name> slot=<n> qty=<after> container=<part pid>`. Nothing is written on the kerbal's recording: his own snapshot carries his side of the move.

**The claim.** Both segment event types are ghosting triggers, so the chain walker's background-event scan claims the member's pid (the member is not in the root lineage: the kerbal does not point at it). The chain's tip is the member's own leaf, so the spawn snapshot is the foreign vessel's own post-transfer form, never the kerbal's; the kerbal stays the tip of nothing and spawns as himself.

**The rewind.** The in-flight commit adoption-stamps the member with the foreign vessel's own pid, and the chain tip spawns with that same pid, but the vessel existed before the tree: a rewind of the tree must leave it in the rewind save for the chain to claim and ghost. So a holder whose spawn pid is the pid of an InventoryTransfer member of its own tree (`ForeignInventoryTransfer.FindAdoptedForeignVesselMember`) neither strips nor keeps its pid in the rewind strip scope (`Rewind strip scope: leave spawned pid=...`, counted as `foreignAdoptedPids=`) and its spawn state resets, so the tip spawns again at the member's end; a member never counts as re-producing another tree's spawned vessel of the same pid; and the OnLoad launch-identity strip leaves the members and their adoption holders out of its match set (`ForeignInventoryTransfer.ExcludeForeignAdoptions`). A pid another holder genuinely re-produces is still stripped. Live finding: EVA-11's reading run 2026-10-15_2302.

**EVA construction on a foreign vessel (todo EVA-CONSTRUCTION-ON-FOREIGN-VESSEL-UNCLAIMED, 2026-10-15).** A part the kerbal attaches to or detaches from a foreign vessel in construction mode changes no inventory slot of that vessel, so it goes through the same gate and the same adoption from its own two stock events. `EVAConstructionModeEditor.AttachInput` fires `OnEVAConstructionModePartAttached(parentVessel, part)` right after `AttachPart` (and `RestoreLastAttachedPart` fires it again when a detach is undone); `DetachInput` fires `OnEVAConstructionModePartDetached(vessel, part)` after `DetachPart`, with the vessel captured before the detach (decompiled KSP 1.12.5). `DetachPart` → `Part.OnDetachFlight` → `Decoupling` → `disconnect` has already removed the held part from `vessel.parts` when the event fires, so the member snapshot taken there is the vessel without the part and needs no frame wait. An attach is a `PartAdded`, a detach a `PartRemoved`, with details `construction part=<name> action=attach|detach pid=<part pid>`; skips log `Foreign construction change not recorded: reason=...`.

**A foreign EVA kerbal (2026-10-23).** Another kerbal standing on EVA is a vessel like any other here: a part taken from or given to his own inventory adopts his kerbal vessel as the member. Its snapshot carries him as the kerbalEVA part's crew, so the cargo flight writes his crew assignment from the transfer on, and the cross-tree claim hand-off (game-actions design 9.3) ends an earlier mission's open-ended hold on him at the transfer, as a dock or a boarding does.

**Not covered.** A part detached from a foreign vessel and then dropped on the ground becomes a new vessel that no recording carries.

---

## 5. Multi-Vessel Recording Sessions

### 5.1 The Problem

KSP gameplay frequently involves multiple vessels that are simultaneously relevant: rendezvous, base construction, orbital assembly. The player switches control between vessels, and the recording must capture all of them at appropriate fidelity so that playback looks correct regardless of camera position.

### 5.2 Recording Session

A recording session is scoped to a single RecordingTree. During a session, the recorder maintains:

1. **The focus track**: Full recording (source=Active) of whichever vessel the player is currently controlling.
2. **Background physics tracks**: Reduced-rate trajectory sampling (source=Background) + all discrete part events for all other vessels within the physics bubble (~2.3km). Sample rate varies by proximity: <200m: 5Hz, 200m-1km: 2Hz, 1km-2.3km: 0.5Hz.
3. **Orbital checkpoints**: Keplerian element snapshots (source=Checkpoint) captured automatically at on-rails transitions, time warp boundaries, and scene changes.

### 5.3 Data Model

The recording session uses the data structures defined in Section 4.3. In summary: the RecordingTree holds a dictionary of Recordings (one per vessel), connected by BranchPoints. Each Recording's TrackSections list contains typed trajectory chunks tagged with a source (Active, Background, or Checkpoint) for diagnostics. The merge algorithm (Section 9) produces a single merged Recording per vessel with non-overlapping TrackSections selected by highest-fidelity-wins.

### 5.4 Focus Switching

When the player switches focus from vessel A to vessel B:

1. Vessel A's active recording transitions to background (or orbital checkpoint if going on rails).
2. Vessel B's background recording transitions to active.
3. The tree's ActiveRecordingId changes.

The focus history is implicit in the sequence of ActiveRecordingId changes — no explicit focus log is stored.

The decision is `FlightRecorder.DecideOnVesselSwitch` ( `TransitionToBackground` / `PromoteFromBackground` ). It has no stop outcome: a focus change never ends the tree's recording session.

**Switch-To and Fly continuation segments.** Taking a vessel over through a stock button (Tracking Station Fly, the Space Center vessel marker's Fly, Map view Switch-To) starts a recording segment for it at once. The trigger is a positive intent armed by the click, never an inference from a focus change. Plan of record: `docs/dev/done/plans/segment-scoped-switch-fly-autorecord.md` .

- **Arming.** `StockActionIntentMarker` is armed only by confirmed stock-UI click handlers: `SwitchIntentTrackingStationFlyPatch` ( `Patches/GhostTrackingStationPatch.cs` ), `Patches/KscVesselMarkerFlyPatch.cs` ( `KSCVesselMarkers.FlyVessel` ) and `Patches/MapFocusObjectOnSelectPatch.cs` ( `MapContextMenuOptions.FocusObject.OnSelect` ; the Postfix refunds the marker when stock's `SetActiveVessel` returns early). The marker carries the target pid, the capture UT, a wall-clock TTL (10 s for the two Fly buttons, 2 s for Switch-To) and `ParsekProcess.ProcessSessionId` . `ParsekScenario` saves it, so a Fly marker survives the scene load; on load the pure `EvaluateStaleness` clears a marker from another process, past its TTL, or from before a UT regression (a quickload).
- **Consuming.** `ParsekFlight.TryConsumeStockActionIntent` runs from `OnVesselSwitchComplete` (Switch-To) and `OnFlightReady` (Fly). The pure `StockActionIntentConsumeDecision.Evaluate` ( `SwitchSegmentConsume.cs` ) re-applies the staleness checks and also refuses a marker for a different vessel, a duplicate of the armed session's target, or one raised during missed-switch recovery; every refusal clears the marker and logs its reason. A target on the surface (PRELAUNCH, LANDED, SPLASHED) is left to the ordinary auto-record trigger, so flying to a parked vessel does not start a segment that would only be discarded as idle.
- **Three branches**, tried in order. (1) **Committed-tree clone:** the vessel is the spawned end of a committed tree; a copy-on-write clone of that tree becomes the live tree ( `TryRestoreCommittedTreeForSpawnedActiveVessel` ) and the segment attaches under it, so the committed original is untouched until the player merges. The same restore also runs with no click (an on-surface target the consume leaves to the trigger, a flight load focused on the vessel, the 1 Hz retry); there it records into the matched recording only when that recording's snapshot (ghost, else vessel) holds every live part persistentId ( `ResumedSpawnPartCoverage` ). A Parsek spawn regenerates them, so a spawned vessel continues in a new segment under the recording instead (entry reason `ResumedSpawnedVessel` , a session with an empty intent id); its part events then resolve against the segment's own start snapshot, and a stock click consumed for the same vessel afterwards adopts that segment rather than adding a second. (2) **Background-member continuation:** the live tree tracks the vessel in its `BackgroundMap` ; the segment continues that member. (3) **Standalone:** otherwise the segment is the root of a new tree.
- **Building.** `SwitchSegmentBuilder` is the pure tree mutation. `ResolveSwitchContinuationParent` walks read-only from the vessel's recordings forward to terminal leaves carrying the same pid; `CreateSwitchContinuationSegment` creates the recording, stamps `Recording.SwitchSegmentSessionId` , attaches it under the chosen leaf with a `VesselSwitchContinuation` branch point (none when standalone) and makes it the tree's active recording. The live wrapper flushes the parent's background track before and removes the vessel from the background map after.
- **Session.** A started segment arms a `SwitchSegmentSession` (session id, tree, parent and segment recording ids, source and focused pids, switch UT, entry reason). It is owned by `ParsekScenario` and saved with the game, so the scoped Discard below stays correct across a save and reload. A successful merge clears it.

- **Scoped Discard.** With a session armed, Discard at the merge dialog removes only the segment: `RecordingStore.TryDiscardActiveSwitchSegmentAttempt` deletes the segment recording and its whole topological subtree (debris, EVA and dock children made during the segment), their branch points, game-state events and sidecars, and the session marker; every committed recording is kept. A segment in a committed-tree clone drops the clone; a segment in a pending tree prunes the tree in place. Without a session, Discard falls back to the whole pending tree (`MergeDialog.MergeDiscardRanToCompletion`).
- **No-op auto-discard.** A resumed segment that changed nothing is dropped rather than prolonging the ghost state. `SwitchSegmentNoOpClassifier.IsNoOpSegment` keeps a segment that has descendants, a destroyed vessel, a meaningful part event (structural, deploy, parachute, robotic, inventory, dock, or engine / RCS at positive throttle; shutdown and zero-throttle seeds are inert here, deliberately unlike `RecordingOptimizer.IsInertPartEventForTailTrim`), a segment event other than a time jump, a flag plant, a dock target, a non-boring track section (atmospheric flight, powered flight, surface motion, an approach), or an orbit change beyond small element tolerances; when in doubt it keeps. At scene exit `SceneExitInterceptor.TryAutoDiscardNoOpSwitchSegment` tears down a no-op standalone segment's tree or reverts a committed-clone segment to the committed original; a segment of a background member of a still-live tree takes the normal commit.
- **Pre-switch Merge / Discard dialog.** The Map Switch-To Prefix may open `MergeDialog.ShowPreSwitchDecisionDialog` before stock switches, gated by the pure `MapFocusObjectOnSelectPatch.DecidePreSwitchDialogAction`. **Case A:** a session is armed and the new target is a different vessel; Merge commits the prior session, Discard scope-discards it. **Case B:** no session is armed, a recording is live, and the target is either unloaded (a far Switch-To reloads the scene through `FlightDriver.StartAndFocusVessel`, which bypasses the scene-exit merge prompt) or a loaded vessel of a separate committed tree (whose clone restore needs no live tree); Merge commits the live tree, Discard discards it. Both buttons then arm a fresh intent and call `FlightGlobals.SetActiveVessel`; there is no Cancel. A re-click on the same target and a re-entry while the dialog is open are filtered by the same predicate.

### 5.5 Scene Changes

When the player leaves the flight scene, all vessels go on rails and orbital/surface checkpoints are captured automatically. When the player returns, new checkpoints are captured and physics sampling resumes. The absence of trajectory points between checkpoints IS the gap, and playback propagates from checkpoints during these intervals.

### 5.6 Time Warp Boundaries

Every time the player enters or exits time warp, Parsek snapshots the orbital elements of all vessels involved in the current recording session. This creates clean reference points for orbital propagation during playback and prevents drift accumulation over long time periods.

---

## 6. Segment Taxonomy

Every vessel segment carries two classification axes that are independent of each other.

### 6.1 Axis 1: Environment

Determined automatically at recording time by vessel state. Changes trigger track section boundaries.

| Environment | Trigger | Sample Rate | Notes |
|---|---|---|---|
| ATMOSPHERIC | Below atmosphere ceiling | High (~50 Hz physics tick) | Chaotic trajectory (drag, lift, heating). No analytical reconstruction possible. |
| EXO_PROPULSIVE | Above atmosphere AND engine producing thrust | High (~50 Hz physics tick) | Thrust is player-controlled, trajectory not Keplerian. |
| EXO_BALLISTIC | Above atmosphere AND no engines producing thrust | Low (orbital checkpoints) | Keplerian, analytically propagable. |
| SURFACE_MOBILE | Landed/splashed AND ground speed > 0.1 m/s | Medium (~5-10 Hz) | Rover driving, vessel repositioning. |
| SURFACE_STATIONARY | Landed/splashed AND ground speed < 0.1 m/s for >3s | Minimal (~0.1 Hz or checkpoint) | Parked vessel, base module. Body-fixed coordinates. |

| Environment | Trigger | Sample Rate | Notes |
|---|---|---|---|
| APPROACH | Airless body AND below the approach-altitude threshold AND not on the surface | High (~50 Hz physics tick) | Low-altitude descent/approach over airless terrain (powered or coasting; engine on/off does not fragment the segment). Load-bearing in the optimizer split predicate. |

Transitions use hysteresis: 1 second for thrust toggle (prevents rapid toggling during pulsed burns), 3 seconds for surface speed threshold.

### 6.2 Axis 2: Reference Frame

| Reference Frame | When Used | Position Data | Playback |
|---|---|---|---|
| ABSOLUTE | Default for active flight | Body-fixed coordinates (body, lat, lon, alt, attitude) | Ghost placed at recorded coordinates. |
| RELATIVE | Close to an anchor vessel (a pre-existing persistent vessel, or a parent during separation) | Anchor-local metre offsets overloaded into the `latitude` / `longitude` / `altitude` fields of each `TrajectoryPoint` (not body-fixed lat/lon/alt); parent-anchored recordings also carry a body-fixed primary surface (see note) | Non-loop sections resolve against a recorded anchor trajectory; loop sections resolve against the live anchor PID. |
| ORBITAL_CHECKPOINT | On-rails coasting, no physics | Keplerian elements at discrete timestamps | Ghost position computed by Kepler propagation to current UT. |

**RELATIVE contract details:** There is one current RELATIVE contract; the legacy v5 world-offset RELATIVE path was purged in PR #916. A `TrackSection` carries BOTH `anchorVesselId` (live-PID anchor, used by loop playback) and `anchorRecordingId` (recorded-anchor trajectory, used by non-loop flight/map/KSC playback). The anchor-local offset is stored as metres along the anchor's local axes in the misleadingly named `latitude` / `longitude` / `altitude` fields, so those values commonly fall outside normal lat/lon ranges - any reader must resolve the section's reference frame before interpreting them. For parent-anchored recordings - genuine debris AND controlled-decoupled children ( `IsDebris=false` probes/landers that come off a parent through a decoupler) - the PRIMARY playback surface is `TrackSection.bodyFixedFrames` (full body-fixed world-coordinate `TrajectoryPoint`s); the anchor-local offsets in `frames` are the secondary surface for loop-anchored chains and diagnostics.

**Docking approach rule:** Any approach within docking range (~200m) of a pre-existing vessel uses RELATIVE reference frame to ensure positional accuracy for visually correct docking playback.

### 6.3 Combined Example

A cargo delivery to a Mun base — one VesselSegment (no staging or docking), nine TrackSections:

```
Recording: "Cargo Run Alpha" (single VesselSegment, 9 TrackSections)

Track 1: [ATMOSPHERIC + ABSOLUTE]         Kerbin launch through atmosphere. ~50 Hz.
Track 2: [EXO_PROPULSIVE + ABSOLUTE]      Gravity turn completion and orbital insertion.
Track 3: [EXO_BALLISTIC + ORBITAL_CKPT]   Kerbin parking orbit coast. Checkpoint per orbit.
Track 4: [EXO_PROPULSIVE + ABSOLUTE]      Transfer burn to Mun.
Track 5: [EXO_BALLISTIC + ORBITAL_CKPT]   Transfer coast.
Track 6: [EXO_PROPULSIVE + ABSOLUTE]      Munar orbit insertion and deorbit.
Track 7: [EXO_PROPULSIVE + ABSOLUTE]      Powered descent. ~50 Hz.
Track 8: [EXO_PROPULSIVE + RELATIVE]      Final approach within physics bubble of base.
Track 9: [SURFACE_STATIONARY + RELATIVE]  Landed near base. Minimal sampling.
```

---

# 7. Background Vessel Recording

## 7.1 Principle

When the player controls one vessel, all other vessels within the physics bubble (~2.3km) are actively simulated by KSP's physics engine. Parsek records these background vessels to ensure playback looks correct regardless of which vessel the camera follows.

## 7.2 Background Trajectory Recording

Background vessels are sampled at reduced rate compared to the focused vessel:

| Distance to focused vessel | Sample Rate |
|---|---|
| < 200m (docking range) | ~5 Hz |
| 200m - 1km | ~2 Hz |
| 1km - 2.3km (physics edge) | ~0.5 Hz |

## 7.3 Background Part Event Recording

**All discrete (visual) part events are recorded for ALL vessels in the physics bubble**: engine ignition/shutdown, staging, decoupling, parachutes, landing gear, solar panels, antennas, cargo bays, fairings, lights, RCS, EVA, etc.

High-frequency continuous events (engine gimbal angles, control surface deflections, throttle) are captured ONLY for the focused vessel.

## 7.4 Structural Events for All Physics-Bubble Vessels

Events that affect the DAG structure (staging, decoupling, docking, undocking) are captured for all vessels in the physics bubble. A background vessel that stages while the player is focused elsewhere produces a proper SPLIT event in the recording tree. Debris children from intentional splits get a TTL: recording stops after `BackgroundRecorder.DebrisTTLSeconds` (60 seconds), or on crash/destruction, or when leaving the physics bubble. Rapid-fire crash fragments are coalesced into BREAKUP events and not individually tracked.

---

# 8. Orbital Checkpoint System

## 8.1 Purpose

For vessels on rails (outside the physics bubble, during time warp, or across scene changes), full trajectory recording is impossible and unnecessary. Parsek records Keplerian orbital elements at discrete points and propagates analytically between them.

## 8.2 Checkpoint Data

Checkpoint data is stored as `OrbitSegment` and `SurfacePosition` values (defined in Section 4.3). In summary:

```
OrbitSegment (used for orbital checkpoints)
  startUT, endUT, inclination, eccentricity, semiMajorAxis,
  longitudeOfAscendingNode, argumentOfPeriapsis, meanAnomalyAtEpoch,
  epoch, bodyName, orbitalFrameRotation, angularVelocity

SurfaceCheckpoint
  ut, body, latitude, longitude, altitude, heading
```

## 8.3 When Checkpoints Are Captured

- Session start (all on-rails vessels)
- Focus switch (vessel going on rails)
- Time warp enter/exit (all session vessels)
- Scene change (all session vessels)

- SOI transition
- Periodic safety net: every 3-5 orbits for long coasts

### 8.4 Playback from Checkpoints

The ghost's position between checkpoints is computed by Keplerian propagation from the nearest prior checkpoint. If a ghost's propagated orbit intersects terrain during a gap, the ghost follows the orbit into the ground — this accurately represents what KSP computed.

---

## 9. Multi-Vessel Merge

### 9.1 The Problem

A recording session involving multiple vessels produces overlapping trajectory data. This must be merged into one continuous track per vessel for the committed timeline.

### 9.2 Merge Rule: Highest Fidelity Wins

For any vessel at any moment in time, use the best available data source:

```
Priority 1: Active focus recording (full trajectory + events)
Priority 2: Background physics recording from nearest focused vessel
Priority 3: Orbital checkpoint propagation
```

### 9.3 Merge Procedure

For each vessel: collect all data sources, select highest-priority source at each UT interval, stitch into one continuous track. Source-switch boundaries use snap-switch (no interpolation) — a 0.5s crossfade at orbital velocities (2200 m/s in LKO) would create 1100m of fake trajectory. The position discontinuity magnitude is logged at each boundary.

### 9.4 No Circular References

At any given UT, exactly one vessel is the active focus (the scene origin). All other positions derive from that perspective or from orbital mechanics. There is no circularity because focus is a total order.

### 9.5 Committed Timeline Output

After merge, each vessel has a single clean Recording with non-overlapping TrackSections covering the full session, each tagged with source (Active/Background/Checkpoint) for diagnostics. This is what gets stored in the timeline and used for all subsequent playback.

---

## 9A. Chain Segmentation and Recording Optimizer

Section 4.4 defines segment boundaries within the recording tree (structural splits/merges only). This section covers the **chain segmentation** system and the **RecordingOptimizer** that merges or splits committed chain segments at save/load time.

> **Note (v0.8.0, T56):** Chain mode (standalone single-vessel recordings with eager boundary splitting) was removed. All recordings now use tree mode. The boundary detection code (9A.2) is retained in `FlightRecorder` but its flags are always suppressed because `activeTree` is always non-null. Chain segments are now produced exclusively by the RecordingOptimizer's post-commit split pass (9A.5). Sections 9A.1--9A.4 describe the original chain mode architecture for historical reference; section 9A.5 (the optimizer) remains current.

### 9A.1 Motivation (historical)

> Chain mode was removed in v0.8.0 (T56). This section describes the original motivation for chain segmentation, which is now handled entirely by the RecordingOptimizer's post-commit split pass (Section 9A.5).

Chain recordings (the former single-vessel recording mode) produced one Recording per flight. A long flight that traversed multiple environments (atmosphere → vacuum → different SOI → landing) resulted in a single monolithic segment. This made it impossible for the player to selectively loop or configure playback for individual flight phases (e.g., loop just the atmospheric ascent, or hide the boring transfer coast).

Chain segmentation automatically split the recording at environment boundaries so each flight phase became a separate, independently configurable Recording linked by a shared `ChainId`.

Tree recordings (now the only mode) do not use eager chain segmentation — they keep all data in a single Recording with multiple `TrackSections`. Environment changes are tracked internally via the TrackSection `environment` field and the session merger (Section 9) handles fidelity resolution. The RecordingOptimizer's split pass (9A.5) produces chain segments post-commit.

## 9A.2 Chain Segment Boundary Conditions

Seven conditions trigger a chain segment commit-and-restart. The first three are environment boundaries; the rest are vessel lifecycle events.

#### A. Atmosphere Boundary (atmospheric bodies only)

Detected per-frame by `FlightRecorder.CheckAtmosphereBoundary()`. Fires when the vessel crosses the body's `atmosphereDepth` altitude.

**Hysteresis** (prevents rapid toggling near the boundary):

- **Time**: 3.0 seconds sustained on the new side
- **Distance**: 1,000 meters past the boundary

Both conditions must be met simultaneously. If the vessel drifts back before both thresholds are met, the pending detection resets.

**Phase tags**: Outgoing segment tagged `"atmo"` (was in atmosphere) or `"exo"` (was above atmosphere).

#### B. Altitude Boundary (airless bodies only)

Detected per-frame by `FlightRecorder.CheckAltitudeBoundary()`. Only active when the body has no atmosphere. Fires when the vessel crosses the **approach altitude threshold**.

**Threshold computation** (`FlightRecorder.ComputeApproachAltitude`):

1. **Primary**: KSP's `timeWarpAltitudeLimits[4]` (the 100x warp limit) — this is KSP's own definition of "close enough that fast warp is dangerous" and adapts to modded planets automatically.
2. **Fallback**: `body.Radius * 0.15`, clamped to `[5,000m, 200,000m]`.

**Hysteresis**:

- **Time**: 3.0 seconds sustained
- **Distance**: `max(1000, threshold * 0.02)` meters past boundary

**Phase tags**: `"approach"` (descended below threshold) or `"exo"` (ascended above).

#### C. SOI Change

Detected by `FlightRecorder.OnSoiChange()` when a vessel transitions between celestial bodies' spheres of influence. The current segment is committed immediately (no hysteresis — SOI changes are discrete events). Phase tag is derived from the **departing** body's environment at time of exit.

#### D. EVA Exit (Vessel → EVA)

When a Kerbal goes EVA, the recording tree branches at an EVA branch point: the Kerbal becomes the active child recording and the vessel keeps recording as a background child of the same tree.

#### E. EVA Boarding (EVA → Vessel)

When a Kerbal boards a vessel, the EVA segment commits and a new vessel recording begins.

#### F. Dock/Undock

Handled by `ParsekFlight.HandleDockUndockCommitRestart()` with four sub-scenarios (initiator dock, target dock, undock-stay, undock-switch). Each causes the current segment to commit. On undock, the sibling vessel gets a **ghost-only continuation** (`ChainBranch=1`) — it is adaptively sampled but never spawns as a real vessel at playback time.

#### G. Vessel Switch During Active Chain

If the player switches vessels while a chain is recording, the entire chain terminates (`HandleVesselSwitchChainTermination`). No continuation is possible because the focused vessel has changed.

#### Not a Boundary

Staging and part joint breaks are recorded as `PartEvents` within the continuing segment. They do **not** cause chain splits.

### 9A.3 Tree Mode Suppression

When recording in tree mode (`activeTree != null`), atmosphere, altitude, and SOI boundary splits are **suppressed**. The boundary flags on the FlightRecorder are cleared (`"Boundary flags cleared (tree mode bypass)"`), and the data stays in the single tree Recording. The environment changes are captured as TrackSection boundaries within that Recording instead.

This is because tree recordings use the session merger (Section 9) for multi-source data and the RecordingOptimizer's split pass (Section 9A.5) post-hoc if the user later wants to break them apart.

### 9A.4 Chain Structure

Each chain segment is a separate `Recording` linked by:

- **`ChainId`** — shared GUID identifying the chain
- **`ChainIndex`** — 0-based sequential position (reindexed by `StartUT` after optimizer passes)
- **`ChainBranch`** — 0 = primary (playable, spawnable), >0 = parallel ghost-only (e.g., undock continuation)

The **boundary anchor** ensures seamless stitching: the first TrajectoryPoint of each new segment is copied from the previous segment's last point, guaranteeing zero positional discontinuity at chain boundaries.

Mid-chain segments have their `VesselSnapshot` nulled (ghost-only playback) — only the final segment in the chain retains the full vessel snapshot for spawn-at-end.

### 9A.5 Recording Optimizer

`RecordingOptimizer.cs` contains pure static functions that merge or split committed chain segments at save/load time. This is a **post-hoc cleanup pass** — it operates on already-committed Recordings.

#### Auto-Merge: `CanAutoMerge(a, b)`

Two consecutive chain segments can be merged if **all** of these conditions hold:

1. Neither is null
2. Same `ChainId` (non-empty)
3. Both have valid `ChainIndex` (≥ 0)
4. Consecutive: `b.ChainIndex == a.ChainIndex + 1`
5. Both on primary branch (`ChainBranch == 0`)
6. No `ChildBranchPointId` on A (no branch point between them)
7. Same `SegmentPhase`
8. Same `SegmentBodyName`
9. Neither has ghosting-trigger events (checked via `GhostingTriggerClassifier`)
10. Neither has `LoopPlayback` enabled
11. Both have `PlaybackEnabled == true`
12. Neither is `Hidden`
13. Both have default `LoopIntervalSeconds` (10.0)
14. Both have default `LoopAnchorVesselId` (0)
15. Same `RecordingGroups` (ordered list equality)

The purpose: if two consecutive segments have the same environment, same body, and neither has been customized by the user, they are redundant splits and should be re-merged into one.

**Merge operation** ( `MergeInto` ): Points concatenated, events merged and re-sorted by UT, TrackSections concatenated, VesselSnapshot and TerminalState inherited from the absorbed (later) segment. Ghost geometry cache invalidated on the result.

**Auto-Split:** `CanAutoSplit(rec, sectionIndex)`

A single Recording can be split at a TrackSection boundary where the environment changes, if:

1. Recording has ≥ 2 TrackSections
2. No ghosting-trigger events anywhere in the recording
3. Both resulting halves are **≥ 5 seconds** long
4. Both resulting halves hold trajectory payload: the recording's actual sampled bounds (points, orbit segments, playable track sections, never an explicit or terminal bound) strictly straddle the cut ( `SplitHalvesCarryPayload` ). The 5-second test reads `StartUT` / `EndUT` , which an explicit bound can carry past the samples (a crash UT after the last sample), so without this a cut there left a half with no sample, whose `StartUT` read the 0.0 fallback.

**Split operation** ( `SplitAtSection` ): Points partitioned by UT at the section boundary. Events partitioned (backward loop to avoid index shifting). TrackSections split at the section index. `GhostVisualSnapshot` cloned to both halves. Each half tagged with `SegmentPhase` derived from its first section's environment via `EnvironmentToPhase` :

| SegmentEnvironment | Phase |
|---|---|
| Atmospheric | `"atmo"` |
| SurfaceMobile | `"surface"` |
| SurfaceStationary | `"surface"` |
| Approach | `"approach"` |
| All others | `"exo"` |

**Parent links across a split** ( `SplitParentLinks` ): the original id stays on the first half and the second half gets a new id, so every link that names the recording as the PARENT of something at or after the cut moves to the second half: each `BranchPoint` whose `ParentRecordingIds` lists it with `UT >= cut - 0.0001 s` (any type: the branch point the recording ends at, which also carries `ChildBranchPointId` over, a decouple the vessel flew on past, a dock, an EVA, a ground part, a post-switch launch), and each child recording whose `ParentRecordingId` names it with a link UT at or after the cut (the UT of the branch point that created the child's first chain segment, else that segment's start, so a split child moves whole). An event exactly at the cut belongs to the second half, whose first sample is the cut. The Re-Fly HEAD/TIP split ( `RecordingTreeSplitter` step 2.6) applies the same rule through its rollback ledger. The merge that reverses a split points every link naming the absorbed recording at the survivor. Trees written before the rule are corrected on load by `SplitParentLinks.RepairStaleParents` (after sidecar hydration in `LoadRecordingTrees` and the pending / active tree restores): a link walks forward through the unique next chain segment while the event is at or after that segment's start, never onto a segment the branch point itself created, and stops on an ambiguous or data-less successor; idempotent, one Info line per repaired tree. The repair rewrites ids within the current shape, so it is not a format change and touches no schema generation. The playback anchors ( `ParentAnchorRecordingId` , `TrackSection.anchorRecordingId` ) are frame links, not parent links, and stay on the segment they were recorded against. Readers that decide a later segment's continuation or leafness from its branch points skip one the vessel flew on past ( `SplitParentLinks.IsFlownPastOnLaterSegment` : a later chain segment, at or after its start, not its `ChildBranchPointId` and not at its end UT), which is how those readers saw the branch point when it stayed on the first segment. Readers that walk UP from a child cross a split through the chain predecessor (a later segment has no branch point or parent link of its own): the logistics transport lineage does, and the mission Log lets any later segment own a stage whose branch point still names an earlier segment that ended before it (a repair that had to stop).

**Career results across a split** ( `RecordingStore.RetagLedgerActionsAfterOptimizationSplit` ): three things carry the recording id as their owner, and the split moves each one whose UT is at or after the cut to the second half, with the same `>=` sense as the parent links: the ledger rows ( `Ledger.RetagActionsForSplitSecondHalf` , attribution UT: a death row by its `EndUT` , a KerbalDeath reputation penalty with its paired death), the captured game-state events not yet converted ( `GameStateStore.RetagEventsForSplitSecondHalf` ) and the pending science subjects by capture UT ( `GameStateRecorder.RetagPendingScienceForSplitSecondHalf` , which also retags the cached pending-milestone copies the reward enrichment matches by tag). The last two matter at a fresh commit: the flight commit paths ( `MergeDialog.MergeCommit` , `ParsekFlight.CommitTreeFlight` ; the ghost-only auto-commit outside flight converts first) run the optimizer BEFORE `LedgerOrchestrator.NotifyLedgerTreeCommitted` converts the flight's events into rows, and the converter reads an event's tag as ownership, so without them every result of the flight was booked on the first segment (RETAG-ON-SPLIT-MISSES-LATER-ROWS). A Re-Fly of a later segment carves the earlier chain segments out of its tombstone set ( `SupersedeCommit.IsPreRewindCarveOut` ), so a row is retired by a re-fly only when it sits on the segment that owns its UT.

**Discovery Passes**

`FindMergeCandidates`: Groups committed recordings by `ChainId`, sorts each group by `ChainIndex`, tests all consecutive pairs with `CanAutoMerge`.

`ReindexChain`: after every merge and split, renumbers a chain's branch-0 members from 0 in `StartUT` order (ties by the previous index, then id). A member with no trajectory payload and no explicit start has only the 0.0 fallback as its `StartUT`, so it is ordered by its first track section's start instead, or last with no section: the highest index must stay the chain's end, which the chain-tip walks read.

`FindSplitCandidates`: Scans each committed recording's `TrackSections` for adjacent sections with **different environments**. Tests each boundary with `CanAutoSplit`. Finds **at most one split per recording per pass** — the caller re-scans after each split because indices shift.

`FindSplitCandidatesForOptimizer`: Same as `FindSplitCandidates` but uses `CanAutoSplitIgnoringGhostTriggers` — does not require absence of ghosting-trigger events (engine ignitions, RCS activation, etc.). This is correct for the optimizer because both halves inherit the `GhostVisualSnapshot` and part events are correctly partitioned by `SplitAtSection`. The ghost rendering system handles part events over time. The conservative `HasGhostingTriggerEvents` check remains on `CanAutoSplit` for other callers (ghost chain walker).

**Which boundaries split** (`IsSplittableEnvOrBodyBoundary`, evaluated top-down): (1) a section flagged `isBoundarySeam` (recorder bookkeeping) never splits; (2) a boundary with the same environment class and body is not a boundary; (3) a body change between two coasting Exo sections stays one recording (a transfer coast); (4) any other body change splits; (5) a boundary with Surface on either side splits, except (a) a brief Atmospheric / Approach run (under 120 s) bracketed by Surface on both sides, a hop or bounce, and then (b) the departure from the **leading launch-site Surface run**: when the recording starts on a launch site (`LaunchSiteName` set, which the recorder writes only for a launch start on the pad, the runway or another stock site, or `StartSituation` Prelaunch) and its first sections are Surface on its own start body, the first Surface → non-Surface boundary that is not such a hop is never split, so a launch keeps the time it spent on the launch site with the flight and its recording count never depends on how long ignition-to-liftoff took (owner ruling 2026-10-07). Touchdowns, take-offs from anywhere that is not a launch site (a Mun lander, a field) and every later Surface boundary still split; (6) a boundary where an engine was firing (`ExoPropulsive`) splits; (7) an Atmospheric / Approach ↔ Exo boundary splits unless one side is a brief run bracketed by the other class (a graze). Each suppressed boundary is counted on the per-recording `Split summary` Verbose line (`launchSiteRunKept=` for 5b). Rationale and calibration: `docs/dev/done/plans/optimizer-persistence-split.md`.

The optimizer is wired into save/load (`ParsekScenario`), running merge candidates first (to clean up redundant same-environment boundaries), then split candidates (to break multi-environment recordings into per-phase segments with individual loop toggles). Split recordings share a `ChainId` for UI grouping — they appear as expandable chain blocks in the recordings window.

## 9A.6 Chain Segments Within Tree Recordings

> **Historical note:** Prior to v0.8.0 (T56), chain mode and tree mode were two separate recording strategies. Chain mode split eagerly at environment boundaries during recording; tree mode deferred splitting to the optimizer. T56 removed chain mode entirely — all recordings now use tree mode. The description below reflects the current (tree-only) architecture.

All recordings produce `Recording` objects (same class, no subclasses) with the same playback-relevant fields: `TrackSections`, `Points`, `OrbitSegments`, `SegmentPhase`, `SegmentBodyName`, `GhostVisualSnapshot`. Environment changes within a recording are tracked as `TrackSection` boundaries. The `SessionMerger` resolves overlapping data streams (active/background/checkpoint sources) by fidelity priority.

After commit, the `RecordingOptimizer` split pass (Section 9A.5) breaks multi-environment tree recordings into per-phase chain segments — separate Recording entries linked by `ChainId`, each with its own loop toggle. Split recordings carry both `TreeId` (for tree-level resource tracking) and `ChainId` (for UI grouping as chain blocks).

The `GhostPlaybackEngine` is completely mode-agnostic — it receives `IPlaybackTrajectory` interface references and has zero knowledge of chains, trees, or policy. This clean abstraction boundary means the engine could be extracted as a standalone library.

---

# 10. Looped Recordings

## 10.1 Principle

Looped recording segments are anchored to a real persistent vessel (station, base, etc.). Parsek never modifies the real vessel's state — it only reads its position to compute ghost placement.

## 10.2 Lifecycle

```
Real vessel loads into physics range (or visual range)
  -> Check: any looped recordings anchored to this vessel?
  -> Compute where each loop is in its cycle based on elapsed UT
  -> Spawn ghost(s) at the correct loop phase
  -> Ghosts update position relative to real vessel's actual current position

Real vessel unloads
  -> Destroy all ghosts anchored to it
  -> Store each ghost's current loop phase for resumption
```

## 10.3 Segment Looping

The player marks which segments to loop. Multiple segments can be looped together as a sequence (e.g., powered descent + approach = complete arrival visual). The loop restarts from the first selected segment after the last completes.

## 10.4 Looped Segments Are Always Relative

When a looped segment plays back, it uses the real anchor vessel's current position, not the historical position from recording time. This means zero positional drift regardless of elapsed time. If the anchor vessel no longer exists, the loop is marked as broken.

## 10.5 Validation on Spawn

Before spawning a looped ghost, Parsek validates: does the anchor vessel still exist? Is it on the expected body? Is the docking port still available? If validation fails, the loop is marked as broken in the recordings manager.

## 10.6 Per-Phase Looping

All recordings support per-phase looping. The optimizer split pass (Section 9A.5) breaks multi-environment tree recordings into per-phase chain segments post-commit. The result: separate Recording entries per flight phase, each with its own `LoopPlayback` toggle in the recordings window.

**Auto loop range**: When a recording's loop toggle is enabled, `ComputeAutoLoopRange` trims boring bookends ( `ExoBallistic` orbital coasts, `SurfaceStationary` idle) to auto-select the visually interesting portion. This narrows the loop to the action phase without user intervention.

**Loop range fields**: The Recording carries optional `LoopStartUT` and `LoopEndUT` fields (default `double.NaN` = loop entire recording). When set, the engine's loop math ( `TryComputeLoopPlaybackUT` ) uses these bounds instead of `StartUT` / `EndUT` . The auto loop range sets these automatically; they are also available for manual customization.

**Architecture**: The `LoopStartUT` / `LoopEndUT` fields are exposed via `IPlaybackTrajectory` .
`EffectiveLoopStartUT` / `EffectiveLoopEndUT` static helpers provide cross-validated bounds with inverted-range fallback. No modification to the `TrackSection` struct or the `GhostPlaybackEngine` 's core architecture — only the loop UT computation narrows its range.

---

# 11. Rendering Zones

## 11.1 The Floating Point Problem

KSP uses Unity's single-precision float coordinate system with the scene origin pinned to the active vessel. At large distances, positional precision degrades:

- At 100km: ~0.01m precision (acceptable)

- At 500km: ~0.05m precision (visible jitter)
- At 1000km+: severe jitter, unusable

### 11.2 Zone Definitions

| Zone | Range | Playback | Looped Ghosts |
|---|---|---|---|
| Zone 1: Physics Bubble | 0-2.3 km | Full mesh, part event replay, relative segments active | Full fidelity |
| Zone 2: Visual Range | 2.3-120 km | Ghost mesh, orbital propagation, no part events | Spawned if <50km, simplified |
| Zone 3: Beyond Visual | 120 km+ | No ghost rendered, position tracked for map view only | Not rendered |

### 11.3 Ghost Rendering Rules

- Zone 2 to Zone 3 (outward): mesh disappears.
- Zone 3 to Zone 2 (inward): mesh appears with short fade-in.
- Zone 2 to Zone 1 (inward): switch from orbital propagation to physics-bubble trajectory data. Part events begin.
- Zone 1 to Zone 2 (outward): switch from trajectory to orbital propagation. Part events stop.

### 11.4 Distance-Based Ghost LOD (replaced the soft caps)

There is no ghost-count cap. The original count-based soft caps (reduce fidelity, despawn the oldest loops, collapse to an orbit line past a ghost count) and their settings were removed in 0.8.1 (PR #226) in favour of a per-ghost distance LOD, so cost scales with what is near the camera rather than with how many recordings exist. The thresholds live in `ParsekConfig.DistanceThresholds.GhostFlight`; the per-frame decision is `GhostPlaybackLogic`'s render-tier resolver:

| Distance from the active vessel | Tier |
|---|---|
| Under 10 km (`FullFidelityRangeMeters`) | Full fidelity: full mesh, part events, engine / RCS / reentry FX |
| 10 km to 50 km (`LoopSimplifiedMeters`) | Reduced: a coarse mesh silhouette, FX suppressed; full fidelity returns below 9.7 km (`FullFidelityRestoreMeters`, hysteresis) |
| 50 km and beyond | Mesh hidden; the ghost's logical playback, map presence and CommNet node continue |

A watched ghost (watch mode, 15.10) ignores the LOD and always renders at full fidelity. Hidden-tier ghosts unload their built mesh and rebuild it from the snapshot shortly before they come back into the visible tiers or before an imminent structural part event.

### 11.5 Map View

In map view, KSP uses double-precision coordinates. Parsek can display ghost positions as icons on the map regardless of distance, without jitter. Ghost orbital paths can be drawn on the map for the full duration of their recording.

---

## 12. Interaction with Pre-Existing Persistent Vessels

### 12.1 Core Rule

Parsek never modifies a real vessel's state directly. The only objects Parsek creates and controls are ghost vessels (raw Unity GameObjects) for playback, and replacement real vessels spawned at ghost chain tips (see Section 12.5).

When Parsek needs to prevent the player from interacting with a vessel during a ghost chain window, it **hides** the real vessel (despawning it and replacing it with a ghost) and **spawns a replacement** at the chain tip. It never edits the vessel's orbit, parts, resources, or crew.

### 12.2 Background Tracks of Existing Vessels

When a pre-existing vessel appears in the physics bubble during recording, Parsek records its trajectory as background data. This serves as anchor reference data for relative-frame positioning, fallback ghost data if the vessel no longer exists at playback time, and validation data to check the vessel is approximately where expected.

If the real vessel IS present during playback, Parsek does NOT spawn a ghost for it. The real vessel serves as its own visual presence.

## 12.3 Merge Events with Existing Vessels

When a recording's vessel docks to a pre-existing vessel, the recording tree notes the merge with a `targetVesselId`. The ghost chain model (Section 12.5) resolves how this interaction is replayed when the player rewinds.

## 12.4 The Vessel Interaction Paradox

When a committed recording includes a physical interaction with a pre-existing vessel (e.g., docking to a station), a dependency is created: that vessel must be in a specific state when the ghost interaction occurs during playback. If the player rewinds and modifies the vessel before the recorded interaction, the timeline breaks.

**The Scenario (Three Timeline Runs):**

**Setup:** Station S exists in 100km Kerbin orbit (launched in earlier recording R0). S has one docking port and one probe core.

**Run 1 — Normal play.** The player launches vessel A and docks it to S, then commits this as recording R1.

| UT | Station S | Vessel A | What happens |
|---|---|---|---|
| 0 | real, bare | — | S orbiting from R0 |
| 1000 | real, bare | real | A launches, R1 recording starts |
| 1600 | real, combined S+A | part of S+A | A docks to S, R1 recording ends |

R1 is committed. It covers UT 1000-1600 and contains a MERGE event at UT 1600 targeting S.

**Run 2 — Rewind to UT 500.** R1 is committed and claims S via the merge event. S becomes a ghost until R1 resolves.

| UT | Station S | Vessel A | What happens |
|---|---|---|---|
| 500 | ghost, bare | — | Quicksave loaded, S is ghost because R1 claims it |
| 1000 | ghost, bare | ghost | R1 playback starts, ghost-A appears |
| 1600 | real, combined S+A | part of S+A | R1 playback completes, real S+A spawns |

This works. S is untouchable as a ghost.

**Run 3 — Rewind to UT 500 again (the paradox scenario).** R1 is still committed. The player launches a new vessel B at UT 800 and wants to dock B to S at UT 1200 — before R1 completes.

| UT | Station S | Vessel A | Vessel B | What happens |
|---|---|---|---|---|
| 500 | ghost, bare | — | — | Quicksave loaded, S still ghost |
| 800 | ghost, bare | — | real | B launches |
| 1000 | ghost, bare | ghost | real | R1 playback starts |
| 1200 | ghost, bare | ghost | real, wants to dock | **B cannot dock — S is a ghost** |
| 1600 | real, combined S+A | part of S+A | real | R1 completes, real S+A spawns. B can now dock. |

B cannot interact with S before UT 1600. After UT 1600, S+A exists as real and B can dock to it. The player simply time-warps past the ghost window — zero gameplay cost in stock KSP (no time-dependent consumables). Alternatively, the player uses the relative-state time jump (Section 14.5) to skip past the ghost window while preserving rendezvous alignment.

## 12.5 The Ghost Chain Rule

**When a committed recording contains a physical interaction with a pre-existing vessel, that vessel becomes a ghost from the moment the rewind quicksave is loaded until the recording completes and the final-form vessel spawns as real.** The ghost window starts at whatever UT the rewind loads — not necessarily UT=0.

If multiple committed recordings interact with the same vessel lineage, the ghost chain extends:

| Recording | Event | Ghost chain state |
|---|---|---|
| R1 | A docks to S at UT 1600 | bare-S ghost → S+A ghost |
| R2 | B docks to S+A at UT 2000 | S+A ghost → S+A+B ghost |
| Spawn | — | real S+A+B spawns at UT 2000 |

The real vessel materializes only at the tip of the chain — after the last committed recording that touches the vessel lineage completes. Parsek walks the full chain of committed recordings that reference a vessel's lineage to determine the spawn point.

**Chain walker algorithm:**

```
function ComputeAllGhostChains(committedTrees, rewindUT):
  claims = {}   // map of vesselPID -> list of {treeId, recordingId, branchPointId, ut, type}

  // Step 1: Scan all committed trees for vessel-claiming events
  for each tree in committedTrees:
    for each branchPoint in tree.BranchPoints:
      if branchPoint.Type in {Dock, Board, Undock, EVA, JointBreak}
         and branchPoint.TargetVesselPersistentId != 0:
        claims[branchPoint.TargetVesselPersistentId].add({
          treeId: tree.Id, ut: branchPoint.UT, type: "BRANCH_POINT"
        })
    // Also scan background recordings for ghosting-trigger PartEvents
    for each recording in tree.Recordings where recording is background:
      if recording has any PartEvent with ghosting-trigger type (not LightOn/LightOff):
        claims[recording.VesselPersistentId].add({
          treeId: tree.Id, ut: event.ut, type: "BACKGROUND_EVENT"
        })

  // Step 2: Build chains, sorted by UT
  chains = {}   // map of vesselPID -> GhostChain
  for each (vesselPID, claimList) in claims:
    sort claimList by UT ascending
    chain = new GhostChain(originalVesselPid: vesselPID, ghostStartUT: rewindUT)
    chain.links = claimList
    chain.tipRecording = find the Recording at the last claim's endpoint
    chain.spawnUT = chain.tipRecording.EndUT
    chain.isTerminated = chain.tipRecording.TerminalStateValue in {DESTROYED, RECOVERED}
    chains[vesselPID] = chain

  // Step 3: Cross-tree linking — extend chains across trees
  for each chain in chains:
    tipRecording = chain.tipRecording
    tipVesselPID = tipRecording.VesselPersistentId  // PID of vessel at chain tip
    // Check if any OTHER chain claims this PID
    if tipVesselPID in claims and tipVesselPID != chain.originalVesselPid:
      // Merge into the existing chain for tipVesselPID
      extend that chain with this chain's links
      // The merged chain keeps the absorbed chain's claimed vessels: its tip ends them too,
      // so all of them are held back until the tip (a tug carrying a depot to a station)
    // Also check: does any BranchPoint in another tree have
    // TargetVesselPersistentId == tipVesselPID?
    // If yes, this chain extends through that tree.

  // Step 4: Compute final spawn points
  for each chain in chains:
    walk to the furthest link -> that Recording's endpoint is the spawn point
    chain.spawnUT = furthest link's Recording EndUT
    chain.tipRecording = furthest link's Recording

  return chains

function IsIntermediateChainLink(chains, recording):
  // True if this recording's EndUT matches a non-final link in any chain
  for each chain in chains:
    if recording's EndUT matches a link that is NOT the chain tip: return true
  return false
```

**Output:** A map of `{vesselPID -> GhostChain}` where each GhostChain contains: the original vessel PID, every claimed vessel it holds back (its own key first, then the key, launch guid and first claim UT of each chain folded into it by the cross-tree link), the

ghost start UT, the ordered list of chain links, the tip Recording (which provides the spawn snapshot), the spawn UT, and whether the chain terminates (no spawn). This map is recomputed on every rewind and save load from the committed trees; it is never persisted.

**Outside the flight chain path.** Only the flight scene ghosts a claimed vessel (on flight load, while the chain's spawn UT is ahead). The Space Center and the Tracking Station do not, and a flight load past the spawn UT no longer ghosts the chain, so after a rewind or a load to before the claim the claimed vessel stays live there in its pre-claim form, under its claimed pid (the tip's own pid when the claimed vessel dominated the dock, a new one on undock when it did not, Section 12.8). When the tip's spawn UT then passes (the Tracking Station hand-off, the Space Center end spawn, or the flight leaf spawn), every live vessel carrying one of the claimed pids that end at the tip is replaced, not adopted or left beside the tip: each is removed with `Vessel.Die()` after its crew are taken off and set Available (no recovery, no funds, no ledger row) and the tip spawns once from its snapshot with its identity preserved (Section 13.3). A tip that ends several claimed vessels (a tug docks to a depot, then carries it to a station: the walker folds the station's chain into the depot's) replaces all of them and spawns as their recorded merged form, and flight likewise holds back every claimed vessel of a chain, its key and each folded claim, not only the key. The replacement needs two independent pieces of evidence, for every live claimed vessel: this session saw the playhead strictly before the tip's start (a rewind or a load), and the live vessel was last simulated in physics (`lastUT`) before the chain's last claim, which no vessel that went through every claim was. A vessel the player is flying or recording is never replaced, nor are the claimed vessels when a live vessel already matches the tip's own identity on a pid that is not one of them (the site adopts that one); a stale claimed vessel carrying the tip's own pid is replaced like the others. One live claimed vessel in doubt keeps all of them on the plain adoption. Claimed vessels are counted by live vessel, not by pid: a station claimed again under the new pid it took in a dock it did not dominate is still one vessel, its original pid the live one. A replacement never loses a vessel: they are removed only when the tip can spawn now (not abandoned, out of attempts, held by the terminal-orbit safety, without a snapshot, or retiring in the KSC exclusion zone, where 13.1 adopts the live counterpart), a removal that fails puts back the ones already removed, and if the spawn still yields no vessel every removed one is respawned from the snapshot taken just before its removal. Any doubt keeps the plain adoption. A plain Rewind-to-Launch now despawns the claimed vessel at the rewind in every scene, as this section and 20.3 describe (2026-10-22, todo CLAIMED-VESSEL-DESPAWN-AT-REWIND-OUTSIDE-FLIGHT): its OnLoad removes every vessel an active chain holds back from the rewind's world (`RewindVesselHold`), keeps the rewind save's copy for the flight hold ghost, and shows nothing for it outside flight until the committed flight's own ghost appears (owner ruling 2026-10-22). The stale-vessel replacement above stays for a world that still holds one (a load outside a rewind). Where a pre-claim vessel is still live outside flight, it cannot be disposed of there (GC-2): while the flight load would ghost it (the chain not terminated, its spawn UT ahead, the pid one `ParsekFlight.SelectChainPidsToGhost` returns), the Tracking Station's Recover and Delete and the Space Center vessel marker's Recover are greyed with the reason on their stock tooltips, and the stock handlers behind them refuse it (`ClaimedVesselDisposalGate`, `Patches/ClaimedVesselDisposalPatches.cs`). Recovering it would let the tip spawn its parts a second time and pay for them twice. Fly stays open: a flight load ghosts it as usual. Stock's launch-site clear holds it the same way (GC-9): while it stands on the runway or pad, the launch-site warning in the editor and at the Space Center adds the reason, the editor's dialog offers no Recover, the Space Center's greys its Recover button, and neither recovers it or launches over it (`Patches/ClaimedVesselLaunchSitePatches.cs`).

**Supply-route cargo in the tip snapshot.** The tip snapshot is captured when its mission is committed, so it carries every supply-route crossing that touched the claimed vessel up to the tip's end: deliveries into it, pickups from it, and debits of it as a route origin. A rewind (or a Re-Fly restore, or an in-session load back in time) to before the claim retires the route events after its cutoff, the reverted save restores the origin, and while the claimed vessel is held back the replayed crossings into it are blocked; spawned verbatim, the tip would hold cargo nobody paid for (owner ruling 2026-10-07: subtract it from the snapshot). Each retire (even one that removed no event) hands its removed events to `ChainTipRouteCargo.CaptureRetiredRouteCargo`, and an in-session load whose route reconcile finds nothing after its cutoff (an F9 at the Space Center) calls it with none, which keeps the cargo rows a committed, non-terminated tip carries (the row's route endpoint, read from the pre-rewind route store, or the vessel its writer resolved, is one of the tip's claimed pids or its own pid with a launch guid not conclusively different, and the row is no later than the tip's end), in `RetiredRouteCargoStore` (an additive `RETIRED_ROUTE_CARGO` child of the PARSEK scenario node, saved and loaded with the route store). Each row is tied to the one snapshot that carries it: its tag names the tree, the recording holding the snapshot, that recording's optimizer chain, the capture UT and a fingerprint of the snapshot's resource content, so a later tip of the same tree (a switch continuation flown on from the already adjusted station, a Re-Fly fork) gets none of it, while an optimizer split that moves the snapshot to its later half is followed through the chain. A row is also kept only when it is no later than the snapshot's watermark, the lowest retire or load-back cutoff seen since one first saw that snapshot: a row above it was created in a timeline that branched after the capture. Every site that makes a spawn copy of a recording's snapshot (the shared `VesselSpawner.BuildValidatedRespawnSnapshot(Recording, ...)`, `VesselGhoster`'s chain tip copy, the Space Center end spawn) passes the copy to `ChainTipRouteCargo.ApplyToSpawnCopy`: the rows tagged with that snapshot are undone latest first (a delivery removed, a pickup or origin debit added back), from the claimed vessel's own parts first (`GhostChainWalker.ResolveClaimedPartIds` at each claim), every tank clamped at zero and at capacity. A crossing the current timeline performed again (a cargo row back in the effective ledger, for example into the pre-claim vessel standing live at the Space Center) is paid again and left in; a replay is matched to a stashed crossing by route, stop, row type and nearest UT inside the snapshot's window, never by cycle id, which every rewind rebuilds from the kept rows. The committed snapshot is never changed. Stored-part inventory is not adjusted.

**Supply-route cargo in any end-of-recording snapshot** (todo END-SPAWN-SNAPSHOT-CARRIES-UNPAID-ROUTE-CARGO, owner ruling 2026-10-07). The same holds for an ordinary leaf whose own recorded vessel is a route endpoint (the route-fed vessel's own recording, for example a station flown on from the Tracking Station while a route delivers into it): its snapshot holds the crossings made while it recorded, and after a rewind it spawns from that snapshot. Each retire therefore also tags the rows to every committed final spawn segment (`GhostPlaybackLogic.IsFinalSpawnSegment`, not debris, spawnable or no terminal) whose vessel is POSITIVELY a stop or non-KSC origin of a committed or dormant route: pid equal, then the launch guids decide when both are known, else both root part flightIDs must be known and equal, never a bare pid (`ChainTipRouteCargo.IsPositiveEndpointMatch`; the row carries the endpoint's root part flightID). A recording that is already a chain tip keeps its one chain identity, and a spawn builds exactly one identity (`BuildSpawnIdentity`: the chain tip's, else the leaf's), so it is adjusted once. Tags, watermarks, fingerprints and the replay rule are the chain tip's. The capture UT of a snapshot is the recording's end before any tail trim: `RecordingOptimizer.TrimBoringTail` moves the end and the spawn earlier and keeps the snapshot, so the trim pass notes the pre-trim end against the snapshot's tree and fingerprint (`ChainTipRouteCargo.NoteTrimmedSnapshotCapture`, `CAPTURE` entries beside the stashed rows, only while a route exists) and every capture bound reads the later of the end and that note (`SnapshotCaptureUT`). A crossing in the trimmed tail is thus taken out of the early spawn copy and replayed live by the route after the spawn, paid once; a spawn that comes after such a replay leaves it in. The three copy sites above already carry every end-of-recording spawn (flight leaf, Tracking Station hand-off, deferred spawn queue, Real Spawn Control's warp, Space Center); `ChainTipRouteCargoTests.EveryVesselMaterializer_IsAKnownSpawnCopyOrRestoreSite` pins each production `RespawnVessel` / `SpawnAtPosition` caller so a new spawn path must choose.

## 12.6 Ghosting Trigger Taxonomy

**A committed recording forces ghosting on a pre-existing vessel if and only if the recording contains a recorded event that changes that vessel's physical state.**

**Physical state changes (TRIGGER ghosting):**

*Structural:*

- Docking to vessel (MERGE)
- Undocking from vessel (SPLIT)
- Part destruction on vessel (collision, explosion, overheat)
- Crew transfer to/from vessel (kerbal = vessel; boarding = merge, EVA = split)
- Claw/AGU grab (structural MERGE)
- EVA construction: part added to or removed from vessel (a stored part moved into or out of a foreign vessel's inventory, and a part attached to or detached from a foreign vessel in construction mode, are recorded on that vessel's adopted member, Section 4.12)

*Orbital / positional:*

- Engine burns (changes orbit)
- RCS translation (changes position/orbit)
- Staging (fires decouplers, changes mass distribution and trajectory)

*Part state:*

- Deploying/retracting parts (solar panels, radiators, antennas, landing gear)
- Running/collecting science experiments
- Action group triggers that change physical part state
- Resource transfers (fuel, monoprop)

**Does NOT trigger ghosting:**

*Cosmetic:* toggling lights, animations without physical effect.

*Observation:* switching focus to the vessel, switching focus away, camera movement, map view, recording the vessel's background trajectory, SAS mode changes, minor orbital perturbation from physics settling.

**The dividing line:** Would removing this event from the timeline leave the vessel in a different physical state than if it had been left alone? If yes, the event triggers ghosting. If no, it does not.

## 12.7 Looped Recording Interactions

Looped recordings that include interactions with pre-existing vessels follow a different rule:

- The **first run** of a looped recording is the real run — it follows the ghost chain logic described above. The vessel spawns as real at completion of the first run.
- **All subsequent loops** are purely visual. Ghosts appear and disappear on the loop cycle. They do not affect gameplay, do not spawn real vessels, and do not extend the ghost chain.

**Definition of "first run":** The first chronological playback of the recording after its commit point. If the recording was committed at UT=1000, the first playback spanning UT=1000 to 1600 is the real run. If the player rewinds to before UT=1000 and fast-forwards again, the playback starting at UT=1000 is still the first run — "first" is defined by the recording's timeline position, not by how many times the player has rewound. The real run always corresponds to the recording's original UT span. Every subsequent loop iteration is visual only.

**A chain with a looped phase keeps its first run real** (operator ruling 2026-09-27: the per-recording Loop toggle is visual only). Looping ONE phase of a chain (a single chain segment's own loop toggle) does not turn the whole chain into a replay and changes no career state. The chain's first run is the real run: its tip spawns once, when the first run reaches the end of the tip segment, and every later loop replay is ghost-only (the tip's `VesselSpawned` / spawned-pid dedup, exactly as for a standalone looped recording). Mid-chain segments never spawn, looped or not; each scene's chain-tip gate refuses them. So `ShouldSpawnAtRecordingEnd` has no "chain looping" refusal, and the crew side's final-segment notion (`IsFinalSpawnSegment`, used by the KSC end-of-flight retirement) agrees. A looped tip reaches the spawn through the loop first-run seams below; a non-looped tip of a chain whose earlier phase loops reaches it through the ordinary past-end completion. The spawn logs `Chain loop first-run spawn:` once per recording. The crew side follows (`KerbalsModule.ProcessAction`, `CrewRecoveryReservationClose`): a looped segment elsewhere in the chain changes no hold, so a Recovered end frees the crew at its EndUT, an Aboard / Unknown hold ends at a split handoff, a recovery or a KSC retirement, and a death follows the stock respawn (this retires the 2026-09-26 looping-chain-death rule). The looped segment's own crew rows hold like any segment's: a looped recording, standalone or in a chain, holds its crew for its real first run exactly like a non-looping recording, and the loop replays reserve nobody (LOOP-RECORDING-CREW-NEVER-RESERVED, 2026-10-01). Being part of a looping chain still marks every segment looping-like for the replay-scope render gate (they render regardless of the history scope; their spawn does not). A mission loop over a chain-split tree sets no per-recording toggle, so it was never affected. A Rewind-to-Launch still keeps a chain target's vessel blocked (#573 lifts only standalone targets).

**The first run of a mission loop plays as a normal ghost** (operator ruling 2026-09-24). A mission loop's copies start at its phase anchor: the UT the loop was enabled at, never earlier than the recorded span's end (`MissionLoopUnitBuilder` step 7b-i), or later when the periodicity phase lock snaps it to a launch window. Before that anchor no copy exists, so each scene publishes only the loop units that are live at the current UT (`LoopUnitSet.LiveAt`, via `MissionLoopUnitBuilder.ResolveLiveUnits`) and a unit before its anchor is left out. Its members are then ordinary recordings in every scene (flight engine, Space Center, Tracking Station, map presence, the watch camera), so the first run is drawn exactly like a non-looping recording and its end takes the ordinary completion and spawn. From the anchor on the unit owns the members: the engine rebuilds the slot's ghost for loop cycle 0 (an ordinary ghost carries no loop cycle), so there is never a second ghost. A clock that jumps straight past the anchor never ran the ordinary path; for that case the loop first-run spawn seams (`TryQueueLoopFirstRunSpawn` in flight, `TryLoopFirstRunSpawnKsc` at the Space Center) still spawn the first run once.

## 12.8 Undocking Scenarios

Undocking follows the same ghost logic as docking. If R1 involves undocking a module from S (a SPLIT event), S's physical state changes. Therefore S is a ghost until R1 completes, at which point S spawns as real in its post-undocking form (fewer parts). The undocked module also spawns as a separate real vessel. Both products of the split are independently subject to ghost chain rules. If a later committed recording docks something to the undocked module, that module's ghost chain extends independently.

**Which half is the claimed vessel.** The chain walker finds the tip by walking from the last claim to a leaf, and at every split it follows the CLAIMED vessel by its parts, not by its vessel pid. The pid does not track the vessel: a dock keeps the dominant vessel's pid (`Vessel.GetDominantVessel`: higher vessel type, then mass) and an undock gives the departing half a new pid, so after a heavier visitor docks and leaves, the station is the half with the NEW pid. Part persistentIds survive both, so the walker reads the claimed vessel's part set from snapshots the tree already holds (its own recording when it is in the tree, else the merged recording's parts minus the docking partner's) and takes the child that holds those parts; when several children do, the one keeping the current pid (KSP's root-part side), else the one holding most. With no part snapshot (on the claimed vessel, or on the child keeping the current pid) it falls back to the child keeping the current pid. Implemented in `GhostChainWalker.ResolveClaimedPartIds` / `SelectWalkChild`.

**The tip is the last optimizer segment.** The optimization pass that runs on every load and commit cuts a recording at environment and body boundaries into chain segments: the first keeps the recording's id (and so stays the child of the branch point that started it), and the last one carries the vessel snapshot, the terminal state and the branch point that closes the recording. Wherever the walk finds the vessel ending in a recording (no child branch point, or a breakup-continuous one it flew past), it goes on to the last segment of that recording's chain (`EffectiveState.ResolveChainTerminalRecording`, the same hop the Rewind Point slot walk takes) and continues from there. The chain's tip, spawn UT and termination are therefore read from the segment the tip spawn uses, and an earlier segment is never a leaf (`GhostChainWalker.WalkToLeaf`, `IsTreeFullyTerminated`).

**The tip follows a Re-Fly.** A Re-Fly replaces a vessel's flight from a Rewind Point: it cuts the slot's recording at the rewind into a HEAD and a TIP on one optimizer chain, and on merge every recording of TIP's subtree gets a supersede row naming the fork, which no branch point lists as a child. So wherever the walk hops (the child it picks at a branch point, the hop to a chain's last segment) onto a recording a supersede row names, it goes on to the recording that replaced it (`EffectiveState.EffectiveRecordingId`, nested Re-Flies included) and continues from there: the chain's tip, spawn UT and termination are the fork's, and the fork is not suppressed as an intermediate link of the flight it replaced. The hop is taken only when the replacing recording is in the same tree (supersede rows are tree-scoped) and continues the vessel the walk follows: it holds one of the claimed vessel's parts, or, without part data, carries the replaced recording's pid and launch. A row on another vessel of the replaced subtree (a stage it dropped, the merged vessel of a dock it made) names a fork that does not carry the claimed vessel, and the walk stays where it was. A claim made by a dock or undock that only a replaced flight recorded is still counted (todo CHAIN-CLAIM-FROM-SUPERSEDED-RECORDING).

## 12.9 Edge Cases

**12.9.1 — Vessel destruction terminates the chain.** If R1 crashes into S and destroys S's controller, the ghost chain terminates with no spawn. Ghost-S plays the crash visually, then disappears. Any surviving piece that has a controller (e.g. a secondary probe core) is a vessel under Parsek's identity model and spawns as a real vessel at its final recorded position. Pieces without a controller are debris and do not spawn. The ghost plays the breakup animation; controller-carrying pieces transition from ghost to real at the chain tip, debris pieces simply vanish.

**12.9.2 — Spawn-point collision.** At a chain tip, a real vessel spawns at the ghost's position. If the player has parked another vessel at that position, the spawn could cause physics overlap. Resolution: bounding box overlap check at spawn time. See Section 13 (Spawn System) for full details.

**Full spawn collision sequence:**

| Step | What happens |
|---|---|
| UT approaching chain tip | Warning appears: "S+A spawning in Xs — vessel B is Ym from spawn point" |
| UT reaches chain tip | Bounding box overlap check. If overlap: spawn blocked, the tip's ghost is held past its end and continues on its propagated orbit (or holds its landed pose). |
| While blocked | The ghost is drawn at its propagated position every frame; the overlap is rechecked at that position every 1 second, with no time limit. Warning persists. |
| Overlap clears | Spawn real vessel at ghost's current propagated position. Ghost destroyed in the same frame. Warning dismissed. |

**12.9.3 — Independent ghost chains.** If R1 docks A to S1 and R2 docks B to S2, both chains are independent. The player must wait for both to resolve before interacting with S1 or S2. The player should be informed on screen that real vessels only spawn at the end of their timeline.

**12.9.4 — Recording scope and vessel claiming.** A recording is tied to its vessel controller. A recording does NOT claim a pre-existing vessel merely by switching focus to it. Each vessel has its own separate recording timeline. When the player switches control to a different vessel during a recording session, they are switching to that vessel's own recording chain. A recording can only claim a pre-existing vessel through physical interaction (Section 12.6). If the player only observed a vessel without physical interaction, that vessel remains real after rewind.

**12.9.5 — Intermediate spawn suppression and ghost visual transition.** When a ghost chain has multiple links (bare-S → S+A → S+A+B), Parsek suppresses the spawn at intermediate points. At UT 1600, R1 completes and would normally spawn real S+A, but R2 claims S+A's controller (docking B at UT 2000). Parsek detects this: the controller that would spawn is referenced by a later committed recording. The spawn is suppressed, S+A continues as a ghost, and only S+A+B spawns at UT 2000. Rule: before spawning a real vessel at a chain link, check whether any committed recording further down the chain claims that vessel's controller.

**Ghost mesh transition at intermediate links:** When ghost-S reaches R1's completion UT and the spawn is suppressed, the ghost's visual must change from bare-S to S+A (the post-merge form). The mechanism: destroy the current ghost GameObject (bare-S mesh), create a new ghost GameObject from R1's endpoint VesselSnapshot (which contains the S+A part layout). This is a visual transition only — the ghost remains a ghost. The new ghost continues on the chain using R2's trajectory data.

**12.9.6 — Cross-perspective interaction attribution.** A recording is of vessel B. During that recording, B collides with station S and breaks off S's solar panel. The multi-vessel merge algorithm already walks background recordings of all vessels in the physics

bubble, detects the part destruction event on S, and attributes it to S's own recording timeline. S's timeline now contains a ghosting-trigger event. No special mechanism is needed.

**12.9.7 — Single recording with vessel separation.** R1 launches vessel A. A separates into A1 and A2. A1 docks to S1, A2 docks to S2. The DAG handles this naturally — the chain walker follows edges through split/merge nodes. S1 and S2 are ghosted independently.

**12.9.8 — Vessel recovery terminates the chain.** R1 deorbits S and the player recovers it. On rewind, ghost-S plays the deorbit trajectory. At the chain tip, nothing spawns — the vessel was recovered. The game actions system credits recovery funds through the normal earning-action pipeline.

**12.9.9 — Spawn queue and time warp.** When the player time-warps past a ghost chain tip, spawns are deferred to warp exit. All vessels whose chain tips were crossed during warp spawn as real on warp exit. Spawns within the physics bubble block re-entering time warp until resolved. Spawns outside the physics bubble create a ProtoVessel immediately at the chain tip UT — they appear in the tracking station and map view right away without needing the player's physical presence.

**12.9.10 — Self-protecting property.** The ghosting model is self-protecting by construction. Once R1 is committed and claims S, S becomes a ghost. A ghost has no physics, no colliders, no docking ports — the player cannot create a second recording R2 that also modifies ghost-S. The only way to interact with S again is to wait for the chain tip spawn. This guarantees all recordings modifying a vessel lineage are naturally ordered — no branching conflicts are possible.

**12.9.11 — Asteroids, comets, and debris.** Asteroids and comets have no controller — they are debris under Parsek's identity model. However, background recording captures trajectories of all objects in the physics bubble. When a committed recording grabs an asteroid with the AGU (claw), the merge algorithm finds the CLAW merge event and attributes it to the asteroid's timeline. The asteroid is ghosted from rewind until the chain tip. Background recording provides trajectory data; orbital propagation from last known state fills any gaps when the asteroid was outside the physics bubble.

**12.9.12 — Surface base position drift.** KSP's procedural terrain can shift by a few meters between sessions. The terrain correction system (Section 13.6) handles this via recorded ground clearance, terrain raycast at spawn time, and physics settling.

**12.9.13 — KSP load vs Parsek rewind.** On any save load, Parsek re-evaluates all ghost chains using the loaded save's committed recording set and current UT. If the loaded save predates a recording's commit, no ghost chain exists. If it postdates the chain tip, the vessel is already real.

**12.9.14 — Ghost visual identity.** Ghosts must be visually distinguishable from real vessels. Ghost vessels display a floating text label: vessel name, ghost status ("Ghost — spawns at UT=X"), and which recording claims them. EVA kerbals pass through ghosts without interaction — the kerbal clips through the ghost mesh. This is another reason visual distinction matters: without it, the player may EVA a kerbal toward what appears to be a real vessel, only to find the kerbal passes through.

### 12.10 Fallback: Real Vessel Missing

If a recording references a pre-existing vessel that no longer exists (destroyed, recovered, deorbited since the recording was made):

- For background tracks: spawn a ghost from the background track data as a visual placeholder.
- For merge events: the ghost reaches the merge point and despawns. The merge target's absence is logged.
- For looped recordings: the loop is marked as broken (see Section 10.5).

---

## 13. Spawn System

### 13.1 Spawn at Recording End

When a recording reaches its end UT, Parsek checks whether to spawn a real vessel:

- If the vessel doesn't exist in the game world, spawn it from the recording's snapshot.
- If it already exists (from the quicksave), skip — the ghost just despawns.
- If the flight ended parked in the KSC exclusion zone, retire it: no vessel (see below).
- On revert, spawn state is reconciled after vessel stripping (see Section 13.11).

Looping recordings spawn their vessel at the end of the **first** playthrough. Subsequent loop cycles are visual-only ghosts.

**KSC end-of-flight retirement** (operator ruling 2026-09-23). A recording whose vessel ENDS its flight parked in the KSC exclusion zone has ended its flight and is retired: it never becomes a real vessel, in any scene (flight, Space Center, Tracking Station), and its ghost is not held for spawn retries. The zone is the two 50 m circles of #170, around the launch pad centre and the runway's west threshold ( `SpawnCollisionDetector.DefaultKscExclusionRadiusMeters` ); a plane stopped mid-runway is outside it and spawns normally. The predicate is `SpawnCollisionDetector.DecideKscEndOfFlightRetirement` : a non-EVA vessel, an effective terminal of Landed or Splashed (the stamped terminal state, or for an un-finalized recording its snapshot situation, where PRELAUNCH maps to Landed), on the home world, whose end position (the position the spawn would use: the vessel snapshot's lat/lon, or the trajectory endpoint for EVA and breakup-continuous recordings) lies inside a circle. When that position is the snapshot's, the recording's resolved trajectory endpoint must lie inside a circle too (operator edge-case ruling: parked for a while, then moved, is no conflict), so a stale start-of-flight snapshot on the pad of a flight that ended elsewhere is not retired; a dropped in-memory snapshot is re-hydrated from its sidecar before the decision, exactly as the spawn gate does, so the spawn side and the crew side read the same position. Only the FINAL stop of the whole flight counts: the rule applies exactly where an end-of-recording real spawn would otherwise be attempted (a spawn-eligible final segment, chain tip or tree leaf), so a vessel that sat on the pad and then drove or flew off, in the same recording or through a later chain or switch-continuation segment, is not retired, and neither is an intermediate chain link or a non-leaf segment that happens to end on the pad. Every end-of-recording spawn entry point (the flight completion and warp flush, tree leaves, the Space Center, the Tracking Station hand-off, and the chain-tip spawns in flight and during a time jump, which also close the retired chain and remove its map ghost) calls `VesselSpawner.TryRetireEndedFlightAtKsc` after its source-vessel adoption check (a real counterpart that still exists is adopted, never retired) and before any spawn route: it settles the recording as spawned with no vessel ( `VesselSpawned` and `SpawnAbandoned` true, spawned pid 0, the same transient state an abandoned spawn uses, which a rewind or revert resets) and logs one Info line, `[Spawner] Spawn RETIRED for #N (Name): flight ended within KSC exclusion zone (pad|runway) - no vessel lat=... lon=... body=... terminal=... rec=...` . The kerbals aboard are freed at the recording's EndUT as if recovered (see `parsek-game-actions-and-resources-recorder-design.md` 9.3); no recovery funds and no ledger row.

## 13.2 Chain-Aware Spawn

When ghost chains are active, the spawn logic is extended:

```
Recording reaches EndUT
  -> Is this an intermediate chain link?
    -> Yes: suppress spawn, continue ghost chain
    -> No: proceed to spawn
  -> Does the real vessel still exist? Yes: adopt it, no spawn
     (unless it is the chain's claimed original still in its pre-claim form outside the
     flight chain path: replace it, Section 12.5)
  -> Did the flight end parked in the KSC exclusion zone?
    -> Yes: retire it, no vessel, no hold (Section 13.1)
  -> Is spawn blocked by collision?
    -> Same-name blocker: recover it, proceed (#112)
    -> Other blocker: walk back to a clear point at once, proceed (Section 13.7)
    -> Walkback exhausted: abandon the spawn
    -> Clear: proceed
  -> Is this a surface spawn?
    -> Apply terrain correction
  -> Spawn vessel (preserving PID for chain-tip spawns)
```

A blocking vessel never holds the ghost on its own: it is recovered, walked around or the spawn is abandoned. The bounded ghost hold of Section 13.5 covers only a single-point recording and a failed spawn; a flight that ends in the KSC exclusion zone is retired, not held. Ghost-chain tips are the exception: a blocked tip spawn keeps the ghost, walks back only after 5 seconds of a stationary blocker, and is held until the overlap clears (Section 12.9.2, "Chain tips" in Section 13.5).

## 13.3 PID Preservation

For chain-tip spawns, the spawned vessel must preserve the original vessel's persistentId. This is critical for cross-tree chain linking: if R1 docks A to S (PID=100), and the chain-tip spawn produces a vessel with a new PID (e.g. 567), then a later R2 that claims PID=100 cannot link to it. The cross-tree chain breaks silently.

**Fix:** Chain-tip spawns skip PID regeneration. Since the original vessel was despawned during ghost conversion, the PID is free — there is no collision. With PID preservation: R1's spawn has PID=100 (from snapshot), R2's MERGE has target PID=100, chain walker matches correctly, chain extends.

Normal (non-chain) spawns continue to regenerate PIDs as before.

## 13.4 Spawn Collision Detection

The spawn system uses two thresholds:

**Warning radius (200m):** As UT approaches a chain tip, Parsek checks whether any real vessel is within 200m. If yes, a persistent on-screen warning shows distance, vessel name, and countdown. UI-only — does not block spawning.

**Bounding box overlap check (spawn blocker):** At the chain tip UT, Parsek computes the bounding box of the vessel about to spawn (from its recorded part layout) and checks for geometric overlap with bounding boxes of all real vessels in the loaded physics bubble, plus a small padding margin. If overlap is detected, the spawn is blocked.

## 13.5 Ghost Extension

Operator ruling 2026-09-23: this section describes the code's behaviour; the earlier indefinite-extension design was retired.

A blocked non-chain spawn does not normally extend the ghost. The collision check resolves the block at once: a same-name blocker is recovered (#112), any other blocker triggers trajectory walkback to a clear earlier point (Section 13.7), and an exhausted walkback abandons the spawn. In all three cases the spawn is settled, so the ghost ends at the recording's end time as usual.

**KSC exclusion zone: retirement, not a hold** (operator ruling 2026-09-23). A flight that ends parked within 50 m of the launch pad centre or the runway's west threshold is retired at the spawn call (Section 13.1): it is settled with no vessel, so the ghost ends at the recording's end time like any settled spawn. Nothing is held or retried, in flight or during time warp. Until the ruling the zone refused the spawn every frame (#170), which held the ghost for the 5 seconds below, and the Space Center spawned the vessel on the pad with no check at all; the #170 block is gone from the collision check.

The ghost is held past the recording's end time only when the spawn stays unsettled, in two cases:

- **Single-point recording:** an overlap on a recording with one trajectory point, where no walkback is possible.
- **Failed spawn:** the spawn itself fails outright.

The hold is bounded. The ghost stays visible for up to 5 seconds while Parsek retries the spawn every 1 second (`HeldGhostRetryIntervalSeconds = 1`, `HeldGhostTimeoutSeconds = 5`). A retry that succeeds spawns the vessel and releases the ghost. On timeout the ghost is destroyed without a spawn. Destroying a past-end ghost clears its completion mark, so on the next frame the recording completes once more and Parsek makes one last spawn attempt with no ghost to hold (operator ruling 2026-09-23: kept as a last retry, so a blocker that clears just after the window still gets its vessel). If that attempt is refused too, the recording stays unspawned for the rest of the scene.

**Time warp:** a completion during warp defers the spawn (Section 13.8) through the same hold.

**Terminal-orbit spawn safety** (operator ruling 2026-09-27: "Any orbit Parsek calls Orbiting will spawn. A periapsis inside the atmosphere is a decaying flight and ends SubOrbital or Destroyed, never as a refused orbit. If a spawn is still refused for a reason that can never clear, the ghost finishes its replay and disappears at the end of the recording; it is not held."). An Orbiting recording, an EVA kerbal's included, spawns from its recorded terminal orbit propagated to the spawn UT (`VesselSpawner.TryPassTerminalOrbitSpawnSafety`); without recorded orbit data it spawns from its last trajectory point. There is one periapsis line, the periapsis floor of `OrbitClearance`: the atmosphere top on a body with an atmosphere, the body's highest terrain (stock PQS `radiusMax - radius`) on an airless body, sea level when neither is known. Every producer of an Orbiting verdict reads it (commit classification `RecordingTree.DetermineTerminalState`, the unloaded-vessel inference `ParsekFlight.HasStableOrbitEvidenceForTerminalInference`, the scene-exit finalizer's `ApplyPeriapsisFloorToExtrapolatedTerminal`), so a periapsis at or under it is SubOrbital (or Destroyed by the extrapolator) and never reaches the spawn as Orbiting. The spawn check (`TerminalOrbitSpawnSafety.Evaluate`), in order: a non-finite propagated altitude or periapsis, or a periapsis not above the floor, is `CannotSpawnSafely`; a propagated altitude under the atmosphere top plus 5 km (`DefaultSafetyMarginMeters`) with the apoapsis above that margin is `DeferUntilSafe` until the orbit scan's next safe UT (the one remaining use of the margin: the vessel is low in the band right now); otherwise the vessel spawns now, including an orbit that never climbs above the margin and a deferral whose scan finds no next safe UT. A deferral holds the ghost past the timeout until the next safe UT, then the spawn is re-evaluated. `CannotSpawnSafely` (the periapsis refusal, a terminal orbit that cannot be resolved, a spawned terminal-orbit vessel that died, non-finite values) is permanent: the completion does not hold the ghost, a held ghost is released on the next retry tick (`HeldGhostAction.ReleaseCannotSpawnSafely`), and the terminal map presence is not retained, so the ghost ends at the recording's end time. Across a load only the spawn-death abandon persists (BUG-C: a vessel that died on spawn is not re-materialized every scene); every other saved refusal is dropped and re-derived from the recorded terminal orbit on the next attempt.

**Chain tips** (phase 6b-4, Section 12.9.2). A ghost-chain tip whose spawn is blocked by a collision keeps its ghost past the tip UT for as long as the block lasts: the blocker can move away at any time, so the bounded 5-second hold does not apply (`ChainTipBlockedGhost.IsExemptFromHeldGhostTimeout`). The visible ghost is the tip recording's own playback ghost, held by the

policy; each frame `ParsekFlight.UpdateSpawnBlockedChainTipGhosts` draws it where the vessel would appear if the spawn cleared now: an Orbiting tip with a recorded terminal orbit follows that orbit propagated to the current UT (the orbit the blocked-tip spawn resolves), any other tip holds its recorded end pose body-fixed; both register the FloatingOrigin reapply entry. The spawn is retried every 1 second ( `TrySpawnBlockedChain` , which also walks back after 5 seconds of a stationary blocker). The ghost is released the moment the tip has a real vessel (spawned, walked back or adopted), when the chain closes (KSC retirement, chains rebuilt), when the chain is no longer blocked, and whenever the policy ends the hold for its own reasons (superseded, rewind-retired, a permanent `CannotSpawnSafely` verdict, the ghost destroyed); each hold and release is logged `[ChainTipGhost] Blocked chain tip ghost held: ...` / `... released: ... reason=<spawned|chain-closed|unblocked|index-shifted|hold-ended|ghost-gone>` . A spawn that fails outright (not a collision: the tip recording or its snapshot is gone, or the spawn fails at a clear position) ends the collision block and takes the ordinary bounded hold. Walkback exhaustion does not end the hold (operator ruling 2026-10-01: keep the ghost and keep retrying quietly for the rest of the scene; the vessel appears the moment the spot clears, nothing disappears silently). The retry is quiet: every log line on the 1-second retry path is rate-limited per recording, so only a first or changed verdict prints at once. A walkback that finds a clear point whose spawn then fails backs the full-trajectory rescan off to once every 10 seconds of real time ( `VesselGhoster.WalkbackRescanBackoffSeconds` , logged once per chain); the direct retry at the tip position stays at 1 second. The chain status reads `Spawn blocked -- spot occupied, appears when it clears` and the ghost label `Ghost -- spawn blocked` .

## 13.6 Terrain Correction

For surface spawns, KSP's procedural terrain can shift by a few meters between sessions.

**Step 1 — Record terrain height.** For surface-stationary segment endpoints, the recording stores vessel altitude and terrain height as separate values (the `TerrainHeightAtEnd` field). Ground clearance = recordedAlt - recordedTerrainHeight.

**Step 2 — Terrain raycast at spawn.** Before spawning, cast a ray downward from above the recorded coordinates to get current terrain height at that lat/lon on that body.

**Step 3 — Altitude correction.** Spawn altitude = currentTerrainHeight + recordedClearance.

**Step 4 — Physics settling.** A brief physics settling period (a few frames) where landing legs and wheels interact naturally with terrain. This handles sub-meter errors below raycast resolution.

The blocked-spawn resolution of Section 13.5 applies here too. A blocked surface spawn recovers a same-name blocker or walks back to a clear earlier surface point, and the raycast correction runs at the spawn position actually used. If the spawn is held (single-point recording, failed spawn), a retry within the 5-second hold uses the terrain height at that moment.

## 13.7 Trajectory Walkback

The standard spawn collision system assumes the player can move to clear the overlap. This fails when the blocking vessel is immovable infrastructure (surface base, ground-anchored station).

**Resolution:** As soon as the overlap is detected (and a same-name blocker has not been recovered, #112), walk backward along the spawning ghost's recorded trajectory frame-by-frame, checking bounding box overlap at each position. Spawn at the latest non-overlapping position. Recompute orbital elements (or surface coordinates) for the new spawn position. The result: the rover materializes a few meters back from where it originally parked, as if the base grew and the parking spot is now occupied.

**Fallback:** If the entire trajectory overlaps (blocking vessel grew to cover the full approach path), the spawn is abandoned: the recording is marked spawned-and-abandoned and no vessel appears. There is no manual placement UI (operator ruling 2026-09-23). This should be rare: it requires the blocking vessel to have grown enough to cover the entire approach path. A single-point recording cannot walk back at all and takes the bounded hold of Section 13.5 instead.

**Chain-tip carve-out** (operator ruling 2026-10-01). A ghost-chain tip is not abandoned when its walkback is exhausted. It keeps its ghost and keeps retrying quietly for the rest of the scene, and the vessel appears the moment the spot clears; nothing disappears silently ("Chain tips" in Section 13.5). On the chain path the walkback also starts only after 5 seconds of a stationary blocker, not at once. The non-chain abandonment rule above is unchanged.

## 13.8 Spawn Queue and Time Warp

During time warp, vessel spawning is deferred to a spawn queue. When a recording completes during warp, the spawn is queued. The queue is flushed on warp exit. This prevents physics instability from spawning vessels during warp. Visual FX are suppressed above 10x warp, ghost meshes hidden above 50x warp for performance.

Spawn queue blocks re-entering time warp until all spawns within the current loaded physics bubble are resolved. Spawns outside the physics bubble do not block warp.

## 13.9 Loaded vs Unloaded Spawning

**Loaded spawn (vessel inside physics bubble):** Full sequence: ghost replacement, KSC end-of-flight retirement (Section 13.1), bounding box check, blocked-spawn resolution if blocked (recovery, walkback or abandonment, Section 13.5; the bounded 5-second hold only for a single-point recording or a failed spawn), terrain raycast for surface, physics settling.

**Unloaded spawn (vessel outside physics bubble):** Create a ProtoVessel entry in the save data at the correct orbital elements or surface coordinates. No Unity objects, no bounding box check. The vessel appears immediately in the tracking station and map view and propagates on rails like any normal unloaded vessel.

## 13.10 Ghost Conversion

When a vessel must be ghosted:

1. Quicksave loads — real vessel exists.
2. Ghost chain walker identifies the vessel as claimed by a committed recording.
3. Vessel's snapshot is captured (via the existing backup snapshot mechanism).
4. Real vessel is despawned (removed from the game).
5. Until the vessel's first claim, a ghost GameObject is built with the same visual builder that creates recording ghosts (`ParsekFlight.UpdateClaimedVesselHoldGhosts`, pure decisions in `ClaimedVesselHoldGhost`). Its source is the last Rewind-to-Launch's own copy of the vessel when there is one, else the captured snapshot: the Space Center that a Rewind-to-Launch opens reloads persistent.sfs, so the live vessel, and therefore the captured snapshot, is already in its post-claim form there, while the rewind save holds the form before the claim (`PreProcessRewindSave` copies the VESSEL node of every vessel a chain holds back at the rewind's adjusted UT). It sits where that source says: body-fixed at the snapshot's lat/lon/alt when landed, splashed or prelaunch, otherwise propagated on the snapshot's orbit, with the snapshot's surface-relative rotation, and it registers the FloatingOrigin reapply entry. It is hidden beyond the ghost visual range, yields to any recording of the same launch that is inside its playback window (the vessel is never drawn twice), and is destroyed at the first claim, when the chain goes away, and on scene teardown. Each show and release is logged `[ClaimedHoldGhost] Claimed vessel hold ghost shown: ... from=<rewind-save|despawn>` / `... released: ... reason=<claimed|covered-by-recording|chain-inactive|ghosting-ended|scene-teardown|flight-ready-reset>`.
6. From the first claim on, the committed recording that carries the vessel draws it with its own playback ghost. EVA construction on a cargo partner changes that ghost's shape at the moment it happened (PART_ADDED / PART_REMOVED, Section 4.12): the member's ghost snapshot is the vessel just after its first construction change, a later attach appends the part to it and shows it from the attach UT, and a detach hides the part from the detach UT (`ConstructionGhostShape`; the playback events are `InventoryPartPlaced` / `InventoryPartRemoved` on the ghost part). A part re-attached at another place gets a stand-in node of its own (a derived pid with `parsekGhostOf` naming the live part), because ghost parts are named by persistentId.
7. At chain tip: final-form vessel spawns from the tip recording's snapshot (with PID preservation, minus the supply-route cargo a rewind refunded, Section 12.5), ghost is destroyed.

Safety: the quicksave is always available as a full backup. Ghost conversion is wrapped in error handling — if it fails at any step, the vessel is left untouched and current behavior applies (no paradox prevention, but no data loss).

## 13.11 Spawn State Reconciliation After Revert

Spawn tracking uses `SpawnedVesselPersistentId` on each Recording to prevent duplicate spawns — if the PID is non-zero, the spawn system treats the vessel as already spawned and skips it. This creates an edge case on revert:

```
1. Recording reaches EndUT → vessel spawns → SpawnedVesselPersistentId = X
2. Game saves (PID = X persisted in .sfs)
3. Player reverts/rewinds
4. OnLoad restores SpawnedVesselPersistentId = X from save
5. Vessel stripping removes the spawned vessel (PID = X no longer in flightState)
6. PID is still X → dedup check blocks re-spawn permanently
```

The problem: spawn tracking is restored from the save *before* vessel stripping runs, but stripping removes the vessel the PID points to. The recording thinks it already spawned, but the vessel no longer exists.

**Fix:** `ReconcileSpawnStateAfterStrip` runs after all vessel strip operations complete. It collects the set of surviving vessel PIDs from flightState, then iterates all recordings: any recording whose `SpawnedVesselPersistentId` is non-zero but absent from the surviving set has its spawn tracking reset (`SpawnedVesselPersistentId = 0`, `SpawnNeedsRecovery = false`). The ghost replays normally and re-triggers the spawn at EndUT.

The reconciliation is called at both revert sites in ParsekScenario (OnLoad revert path and the rewind path) to cover all vessel stripping scenarios.

---

## 14. Rewind and Timeline Operations

### 14.1 Core Principle

Committed recordings are permanent — they survive rewind. Rewinding moves the player's position on the timeline but does not erase committed recordings. All committed recording ghosts play at their recorded UTs regardless of the player's current position on the timeline.

Each committed recording has an associated quicksave taken at its launch UT. This quicksave captures the game state needed to resume from that point.

**Timeline immutability (append-only):** Committed recordings are not edited or deleted in place. The player's decision at recording end is commit to timeline or discard. Immutability is append-only rather than absolute: a committed subtree can be superseded by a re-flown fork (Rewind-to-Separation, Section 14.9) - the original rows are retained and marked superseded, never mutated or removed. Nothing in the UI deletes a committed recording or wipes the timeline (the Wipe All buttons and the table's delete button were removed in 0.10.5); to stop seeing a recording the player ticks its Archive checkbox in the Recordings tab instead. Deletion is disallowed because it could orphan spawned vessels, break chain continuity, or create inconsistent ghost playback.

### 14.2 Rewind Procedure

The player is at UT=6000 and rewinds to Recording B's launch at UT=2000:

```
1. Load quicksave at UT=2000.
   Vessels spawned by earlier completed recordings ARE in the quicksave.

2. Ghost chain evaluation fires:
   - All committed recordings are scanned for vessel-claiming events.
   - Any claimed pre-existing vessels are ghosted (despawned, replaced by ghost GOs).
   - Chain state is NOT persisted — it is re-derived from committed recordings on every load.

3. Ghost playback resumes from committed recordings:
   - Recordings completed before UT=2000: their spawned vessels are in the quicksave.
   - Recordings in flight at UT=2000: ghosts play at their trajectory positions.
   - Recordings not yet started: no ghosts yet, they appear at their StartUT.

4. Player resumes at UT=2000 with correct state.
```

### 14.3 Spawn at Recording End

At each recording's end UT: if the vessel doesn't exist in the game world, spawn it from the recording's snapshot. If it already exists (from the quicksave), skip. On revert, spawn state is reconciled after vessel stripping (Section 13.11) to ensure re-spawn is not blocked by stale PID references.

### 14.4 Fast-Forward (Time Warp with Ghost Playback)

After rewinding, the player can time warp forward. All committed recording ghosts play at their recorded UTs. The player can drop out of time warp at any point and start flying. Ghosts continue playing around them.

### 14.5 Relative-State Time Jump

Normal time warp (Section 14.4) is the default way to advance time. The timeline plays out: ghosts appear, play their trajectories, despawn or spawn vessels at their endpoints. The player can fast-forward to any point in the future at any time. For most gameplay, this is sufficient.

The time jump is a specialized mechanism for a specific scenario: **the player has set up a multi-vessel rendezvous approach to a ghosted vessel and needs to skip past its chain tip without losing the alignment.**

**The Problem:** The player rewinds to T0. Ghost-S is in the physics bubble, chain tip at T1. The player maneuvers vessel A into a rendezvous approach — 80m from ghost-S, velocities matched, docking alignment set up. To interact with S, the player must

advance UT past T1. Normal time warp propagates A and ghost-S on their independent Keplerian orbits. Since they are 80m apart, their orbital elements differ slightly. Over the warp interval, differential drift separates them — potentially by hundreds of meters. The carefully set up rendezvous is destroyed.

**The Solution:** A time jump is a discrete UT skip (not a warp) that advances the game clock while keeping every vessel in the physics bubble at its exact current position. No vessel moves. No intermediate physics simulation occurs. Orbital epochs are adjusted so that Keplerian propagation remains consistent with the new UT.

**The Operation:**

1. **Snapshot the bubble.** For every object in the loaded physics bubble (ghosts, real vessels, debris), capture:

```
BubbleSnapshot
  playerVessel:       active vessel
  jumpDelta:          targetUT - currentUT
  objects: [
    {
      id:               object identifier
      position:         current absolute position (body-relative)
      velocity:         current absolute velocity
      attitude:         rotation quaternion
      orbitalElements:  current elements (to be epoch-shifted)
      isGhost:          bool
      chainTip:         UT of chain tip (if ghost)
    }
  ]
```

2. **Jump UT.** Set the game clock to the target UT. No physics frames are simulated between T0 and the target UT.

3. **Adjust orbital epochs.** For every vessel in the bubble (including the player's): keep position and velocity vectors unchanged, but recompute orbital elements to be consistent with the new UT at that position. In practice, this means shifting each orbit's mean anomaly at epoch by the jump delta (targetUT - T0). The orbit shapes (SMA, eccentricity, inclination, LAN, argument of periapsis) remain identical. Only the phase reference changes so that Keplerian propagation produces the correct position at the new UT.

4. **Process spawn queue.** For every ghost whose chain tip was crossed during the jump: destroy the ghost, spawn the real vessel at the ghost's current position (which has not moved), apply bounding box overlap check. If overlap, the blocked-spawn resolution of Section 13.5 applies (recovery, trajectory walkback or abandonment; the bounded hold only in its three cases).

5. **Process game actions.** Trigger recalculation for the new UT - science, funds, reputation, kerbals, facilities, contracts. Same as warp exit, routed through the ledger game-actions system.

6. **Resume physics.** The player is at the target UT, at the same position and velocity relative to the now-real vessel, with the same approach geometry.

**What changes visually:** Planet rotation (Kerbin has rotated to the correct angle for the new UT), sun angle (day/night terminator has moved), star field. All celestial bodies are at their correct positions. These serve as natural visual cues that the jump occurred.

**What does NOT change:** Vessel position vectors, velocity vectors, relative positions within the bubble, relative velocities, vessel attitudes, approach geometry, orbit shapes.

**Surface case:** Both vessels surface-fixed. The body rotates with UT, carrying both. Relative positions preserved automatically (surface-fixed coordinates don't change). The only visual change is the sun angle.

**Atmospheric case:** Epoch-shifting is approximate for atmospheric vessels (trajectory is not Keplerian). Position is preserved exactly, but post-jump orbital elements may not match the atmospheric trajectory that would have occurred. This is acceptable: atmospheric chain tips are rare, relative positions are preserved, and post-jump atmospheric physics resumes immediately.

**Looped ghosts during a jump:** Looped ghosts that are in the bubble stay at their current positions. Their loop phase advances to match the new UT — orbital epochs are shifted like everything else. They continue their loop cycle from the correct phase. No spawn occurs (loops beyond the first are visual only).

**Interaction with active recording:** If the player is recording during a time jump, a TIME_JUMP SegmentEvent is stored with pre-jump and post-jump state vectors. Playback handles the discontinuity as a visual cut — no interpolation across the gap.

## 14.6 Selective Spawning UI (Real Spawn Control)

The player does not need to jump to the furthest chain tip. The Real Spawn Control window (flight scene, Advanced mode; the main window's `Real Spawn Control (N)` button) lists the ghosts the player can turn into real vessels from where they are, each with a one-word status and a `Warp` button that performs the time jump of Section 14.5:

```
Craft                Dist    Speed     Spawns     Spawn date        Status    Actions
Station Alpha        80 m    0.1 m/s   T- 18m 20s Y1, D01, 00:18    Ready     [ Warp ]
Fuel Depot + Cargo   140 m   0.3 m/s   T- 41m 40s Y1, D01, 00:41    Ready     [ Warp ]
Mun Lander           190 m   0.2 m/s   T- 3m 0s   Y1, D01, 00:03    Leaves    [ Warp ]
Rover                210 m   8.1 m/s   T- 1h 5m   Y1, D01, 01:05    Too fast  [ Warp ]  (greyed)
```

**Which ghosts are listed (the spawn distance rule).** A ghost is a candidate when it is active at the current UT, its recording ends in the future and spawns a vessel there (`GhostPlaybackLogic.ShouldSpawnAtRecordingEnd`), it is not an intermediate chain link (`ShouldSuppressSpawnForChain`, Section 12.9.5), and it is within the spawn radius, `ParsekFlight.NearbySpawnRadius` = 250 m, of the active vessel. The radius is the spawn gate itself: a time jump spawns a vessel where its ghost stands, so the window offers only ghosts the player has actually rendezvoused with, and a ghost farther away cannot be spawned from here and is not listed. Relative speed (derived frame-agnostically from the change in the active-vessel-to-ghost separation between two 1.5 s scans) gates the warp at `MaxRelativeSpeed` = 2 m/s; a listed ghost passing faster (up to `MaxListRelativeSpeed` = 50 m/s) stays in the list as a greyed `Too fast` row, because matching speed is what the player can still do. The scan samples ghosts out to 1000 m so a ghost that closes inside the radius is listed with its speed already measured.

**Status and Warp.** `Ready`: inside both gates, Warp jumps to the recording's end, when the vessel spawns here. `Leaves`: the ghost leaves its current orbit (into another body's SOI, onto a new orbit around the same body, or down to the surface) before its recording ends; Warp jumps to just before it leaves (the jump lead), and the hover says it does not spawn here, because the spawn happens later and elsewhere. `Leaving`: that departure is due now (Warp greyed). `Passed`: the spawn time is behind the clock (Warp greyed). `Too fast`: as above. The `Spawns` countdown and `Spawn date` cells show the moment the Warp acts on, and `Warp to Next Spawn` picks the earliest such moment among the rows inside both gates.

**Spawn selection behavior:** the jump spawns the selected ghost's vessel at its chain tip. Any independent chain tips chronologically before the target are also spawned - a ghost cannot remain ghost past its chain tip. Ghosts with chain tips after the target UT remain ghost at their current positions.

**Chronological constraint:** The player cannot jump backward - only forward.

**Chained jumps:** The player can perform multiple jumps in sequence. Each jump epoch-shifts whatever is currently in the bubble (including real vessels spawned by previous jumps).

## 14.7 Time Jump Scenario Walkthroughs

**Simple two-vessel rendezvous:**

| UT | State | Action |
|---|---|---|
| T0=500 | A: real. Ghost-S: 80m away. | Player sets up docking approach. |
| - | - | Player presses Warp on Station Alpha's `Ready` row (T1=1600) |
| T1=1600 | A: real, same position. S: real, still 80m away. | UT jumped. Planet rotated. Nothing in the bubble moved. |
| T1+ | A docks to real S. | Recording R2 starts. |

**Three vessels, player picks the middle one:**

Physics bubble at T0=500: A (real), Ghost-S1 (tip T1=1600), Ghost-S2 (tip T2=2000), Ghost-S3 (tip T3=5000). Player presses Warp on S2's row (T2=2000); S1's tip (T1=1600) is crossed on the way.

| Object | Before jump (T0) | After jump (T2=2000) |
|---|---|---|
| A | real | real, same position, epoch-shifted orbit |
| S1 | ghost (tip T1=1600) | real (tip T1 crossed) at same position |
| S2 | ghost (tip T2=2000) | real (tip T2 = target) at same position |
| S3 | ghost (tip T3=5000) | ghost, same position |

Player docks to S2. Later, if needed, presses Warp on S3's row for another jump.

**Linked chain - player must wait for full chain:**

R1 docks X to S at T1. R2 docks Y to S+X at T2. Chain: bare-S → S+X → S+X+Y. Real Spawn Control lists only the chain tip, the combined vessel spawning at UT=2000.

T1 is NOT offered as a spawn option - intermediate spawn suppression prevents it.

**Surface base approach:**

Rover A is 50m from ghost-base S on the Mun. Chain tip at T1. Player presses Warp on Base S's row. Jump to T1. Both surface-fixed - Mun rotates, sun angle changes, but relative positions identical. Base spawns as real. Rover drives up and docks.

**Jump, then rewind:** Player jumps to T1, S1 spawns real. Player docks A to S1, commits R2. Player then rewinds to T0. Everything resets: S1 is ghost again (R1 claims it), R2 is committed and plays as ghost. The time jump left no persistent state - standard rewind rules apply.

### 14.8 Quicksave Pruning

Deferred. All quicksaves are kept. Pruning is purely storage management with no architectural impact. When implemented: manual control (rewind-protected vs archive), auto-pruning policy (keep N most recent), cascading rewind (if quicksave pruned, rewind to nearest prior quicksave, then fast-forward).

### 14.9 Rewind-to-Separation and Re-Fly (v0.9)

Rewind-to-Separation lets the player roll a mission back to a decoupler/undock separation point and re-fly a fork of a committed subtree. This is the mechanism that makes recording immutability append-only (Section 14.1) rather than absolute: a committed subtree can be superseded by a re-flown fork, with the superseded rows retained and marked, never mutated. The full model - preconditions, crash-recovery journal, and effective-state resolution - lives in `parsek-rewind-to-separation-design.md`; the summary:

- **Invocation** (`RewindInvoker`): a five-precondition gate captures a pre-load reconciliation bundle, copies a Rewind-Point (RP) quicksave to the save root, and on load restores/strips/activates state, writes an atomic provisional, and drops a `ReFlySessionMarker`.
- **Subtree supersede** (`SupersedeCommit`, `RecordingTreeSplitter`): the re-flown fork appends `RecordingSupersedeRelation` rows for the superseded subtree, splitting the closure-root recording into a kept pre-rewind HEAD and a superseded post-rewind TIP. Superseded rows are retained, never deleted.
- **Merge journal** (`MergeJournalOrchestrator`, `LoadTimeSweep`): the merge is driven through crash-recovery checkpoints; an OnLoad finisher rolls back or drives to completion, and a load-time sweep validates the marker and discards zombie provisionals.
- **Effective state** (`EffectiveState`): visibility and ledger reads resolve through the Effective Recording Set / Effective Ledger Set (ERS/ELS), which walk supersede relations so only the winning fork is surfaced.

---

## 15. Ghost Implementation

### 15.1 Ghosts Are Purely Visual

Ghosts are playback-only visual objects. They never become real vessels. They never carry enough state to be "promoted" to real vessels. There is no "take control of a ghost" feature — this is explicitly out of scope due to the complexity of reconstructing full vessel state from recorded data mid-flight.

### 15.2 Raw Unity GameObjects

Ghosts are implemented as raw Unity GameObjects with meshes cloned from part prefabs — NOT as KSP Vessel objects. All visual effects (engine flames, staging, parachutes, solar panels, etc.) are re-implemented by Parsek specifically for ghost playback.

**Advantages:**

- Ghosts are invisible to KSP's vessel management — no tracking station entries (unless explicitly registered), no persistence overhead, no accidental interaction with game systems.

- No KSP per-vessel overhead (orbit calculation, thermal simulation, resource processing).
- Cleaner separation of concerns.
- Safer — no risk of ghost state leaking into the save file or confusing other mods.

**Tradeoffs:**

- All visual effects must be manually re-implemented for ghost playback.
- Modded parts with custom shader effects or animations may not replay perfectly.

**Performance:** Since ghosts are not KSP Vessels, the ghost count limit is driven purely by Unity rendering cost (draw calls, polygon count), not by KSP's vessel management overhead.

### 15.3 Ghost Interaction Rule

Ghosts do not physically interact with anything:

- No collisions with real vessels, terrain, or other ghosts.
- No resource transfer to or from ghosts.
- No docking with real vessels (ghost docking events are visual only — the ghost approaches and despawns at the merge point).
- The player cannot click on or select ghosts for vessel control.
- EVA kerbals pass through ghosts without interaction.

Ghosts are visible scenery. They make the world feel alive but do not participate in game mechanics. The one exception is communications: a ghost relays signal and can be a probe control point exactly as the real vessel would (section 15.6).

### 15.4 Ghost Visual Identity

Ghost vessels display a floating text label: vessel name, ghost status ("Ghost — spawns at UT=X" or "Ghost — loop replay"), and which recording claims them. Labels are visible at all distances within Zone 1 and Zone 2, scaled by distance.

The visual treatment must not obscure the vessel's shape (the player needs to see what vessel it is). The visual treatment should be consistent across all ghost types (chain ghosts, loop ghosts, background trajectory ghosts).

Full visual treatment (transparency, color tint, outline effect) is deferred. The label provides minimum viable distinction.

### 15.5 Ghost World Presence

Ghosts represent vessels that exist in the world pending chain resolution. They participate selectively in game systems:

**Tracking station:** Ghosted vessels appear with a distinct icon. The info panel shows: vessel name, ghost status, chain tip UT, claiming recording.

**Map view:** Ghost orbit line in distinct style (different color or dashed). Vessel marker shows ghost status on hover/click. The player can set a ghosted vessel as a navigation target — approach vectors, closest approach markers, and relative velocity all work normally.

**Loaded ghosts (in physics bubble):** Unity GameObject for visual rendering, plus a CommNet node registered at the ghost's position with antenna specs from the recording data.

**Unloaded ghosts (outside physics bubble):** No Unity object. A CommNet node at the orbital-propagated position, plus a tracking station entry and map view marker. This is the minimum representation for a ghost that is far from the player but still participates in the communication network.

**Implementation: ProtoVessel-based map presence.** Each ghost chain with orbital data gets a lightweight ProtoVessel (single `sensorBarometer` part, `DiscoveryLevels.Owned`). This provides automatic tracking station entries, orbit lines via `OrbitRenderer`, map icons via `MapObject`, and `ITargetable` navigation targeting — all from a single `ProtoVessel.Load()` call. Ghost ProtoVessels are prevented from entering physics simulation via a Harmony prefix on `Vessel.GoOffRails`. They are stripped from saves in `ParsekScenario.OnSave` and reconstructed from recording data on load. A `CommNetVessel.OnStart` patch suppresses the duplicate CommNet node (the ghost's CommNet node is `GhostCommNetManager`'s, section 15.6). Tracking station Fly/Delete/Recover actions are blocked via Harmony patches on `SpaceTracking` methods. All `FlightGlobals.Vessels` iteration sites and vessel GameEvent handlers in Parsek have `GhostMapPresence.IsGhostMapVessel(pid)` guards (27 sites across 9 files). Full design in `docs/dev/done/research/ghost-map-presence-design.md`.

**Ghosts are invisible to the recording system.** Ghost mesh GameObjects are raw Unity objects (not KSP Vessels). Ghost map ProtoVessels ARE in `FlightGlobals.Vessels`, but every recording system path has an `IsGhostMapVessel` guard that excludes them. The background recorder, flight recorder, spawn collision detector, and all vessel event handlers skip ghost ProtoVessels. If a ghost flies through the physics bubble during an active recording session, the recorder does not see it.

## 15.6 Ghost CommNet Relay

**The rule (operator ruling 2026-09-26).** For CommNet, a ghost replaying a committed recording IS the vessel it replays. While that ghost exists in the timeline, its antennas relay signal for live vessels, and it is a probe control point when its recorded crew qualifies, exactly as the real vessel would be at that moment. When the recording ends and the vessel spawns, the real vessel's own stock CommNet node takes over and the ghost node goes away.

**Gameplay scenarios (binding).**

1. **Relay on its way.** The player records a relay flying to Duna and commits it, rewinds, and launches a probe. While the relay's ghost replays the transfer, the probe routes signal through it. When the recording ends at Duna, the relay spawns as a real vessel and stock CommNet carries on without a gap.
2. **Constellation dropped off mid-mission.** A carrier deploys three relays. Each separated relay is its own recording and relays from its separation onward. The carrier relays only with the antennas it keeps.
3. **Crewed ship with a control point.** A crewed Duna ship ghost carries an RC-L01 remote guidance unit and a pilot. A probe the player flies now stays under full control through the ghost even when there is no path to KSC (Duna behind the Sun, or the ghost has only direct antennas).
4. **Mun far side, early career.** A crewed control-point ghost in Mun orbit lets a lander be flown on the far side with no relay network at all.
5. **Jool mothership.** The ghost mothership makes the Jool system flyable for probes launched later, for the length of its ghost window.
6. **Control unit without qualifying crew.** It relays only, no control, as in stock.
7. **Science.** Transmissions route through ghost relays. As in stock, the first hop's relay power sets the transmit rate.
8. **Occlusion.** The node sits at the ghost's recorded position at the current UT, so a ghost relay behind the Mun cannot relay through the Mun.
9. **Deployable antennas.** A deployable antenna relays only while its recorded state is extended. A relay that unfolds after fairing separation joins the network at its recorded deploy event and leaves it at a recorded retract or break.
10. **Destroyed vessels.** A vessel that is destroyed in its recording relays until the recorded destruction and never afterwards.
11. **Loops.** Only the real run relays: the recording's own window up to its spawn. Loop replays and overlap copies are visual repeats and carry no signal.
12. **Hidden ghosts.** A recording whose playback is switched off still relays. Hiding is display only, and the timeline still spawns the vessel.
13. **Recordings no longer on the timeline.** Superseded, rewind-retired and historical-never-replayed recordings get no node. Neither does a recording suppressed because its real vessel already exists; the real vessel carries its own node.
14. **Vessel types stock ignores.** Debris, flags, deployed science parts and space objects get no node, as in stock.
15. **Chain-ghosted real vessels.** A real vessel despawned because a committed future recording claims it keeps relaying until it reappears, with exactly one node for it at any moment: from its despawn snapshot until the first recording that carries it starts (FLIGHT, which holds the snapshot), then through each recording that carries it (docked into another vessel, it relays through that vessel's recording), and between two of them from the end state of the one that ended, at its terminal orbit or landing spot propagated to now, until the next claim or recording of it takes over. The same between-recordings hold covers any recording whose real terminal spawn a later continuation owns (a superseded terminal spawn). A vessel with no known takeover is not held open-ended.
16. **Held ghosts.** A ghost held past its end while its spawn is retried keeps relaying until the vessel spawns or the hold gives up. There is no gap at its end in either scene: a host that ticks slower than every frame (the Tracking Station) works out the hold shortly before the end, so the node stays lit across it.

**Scenes.** FLIGHT (flight view and map view) and the TRACKING STATION, which draws the links. The Space Center and the editors have no consumer, so there are no nodes there. Scenario 13's replay scope is the same in every scene: FLIGHT, the Space Center and the Tracking Station each note every committed recording's playhead every frame, so a Tracking Station entered straight from a loaded save latches a recording whose window is still ahead (a hidden one included), and never one whose start is already behind the playhead.

**The node.** One free `CommNode` subclass per recording, keyed by recording id (never by the craft-baked pid).

- It overrides `position` to return `precisePosition`, because stock's getter reads a transform the node does not have and the CommNet UI would throw.
- `name` is `ParsekGhost:<recordingId>` and `displayName` is the vessel name.
- Both antennas carry a range curve. Stock evaluates it on every link, so a null curve breaks the whole network.
- The position is set in the node's CommNet pre-update hook, at the current UT, from the recorded trajectory. It does not depend on a ghost mesh existing. When the position cannot be resolved (for example a Relative section whose anchor is missing), the node's powers are zeroed for that rebuild; it is never left at a stale position.
- Registration and removal run from the scene host's Update, never inside the pre-update hook, which stock iterates by index.
- On `GameEvents.CommNet.OnNetworkInitialized` (a network reset or a settings change) every node is re-added with its powers recomputed.

**Antenna power.** Derived, never stored; the recording format is unchanged.

- Antennas come from the recording's vessel snapshot. That is the end state, or the start-state visual snapshot when the vessel was destroyed.
- For each antenna, its part prefab supplies power (including upgrades, through `CommPowerUnloaded`), type, combinability, exponent and range curve.
- Powers are combined by stock's algorithm (`CommNetVessel.UpdateComm`), including the relay/transmit split, then scaled by `CommNetParams.rangeModifier`.
- A deployable antenna follows its recorded `DeployableExtended` / `DeployableRetracted` / `DeployableBroken` events; otherwise the snapshot state (`CanCommUnloaded`) decides.

**Control point.** Stock's rule, applied to the recorded vessel:

- a `ModuleProbeControlPoint` that can operate;
- at least `minimumCrew` recorded crew with pilot skill (`FullVesselControlSkill`), looked up by name in the roster (an unknown name does not count);
- multi-hop from the module's own flag.

**Other mods.**

- **CommNet off.** When CommNet is disabled, by difficulty or by RemoteTech (which replaces CommNet), stock creates no CommNet scenario, so no ghost node is registered. Ghosts do not count under RemoteTech's own network.
- **Mods that replace CommNet types** (RealAntennas, CommNetManager-derived networks): ghost nodes register only when the network and range-model types are stock ones; otherwise a one-time log line records the skip.

**Ghost map ProtoVessels** keep their `CommNetVessel` suppressed (`GhostCommNetVesselPatch`). The marker is a single barometer part and must not become a second, zero-power node for the same ghost.

**Live proof by lane (2026-09-26).** Flown and armed: `CN-1-ghost-commnet-relay` / `CN-1T-ghost-commnet-relay-ts` (FLIGHT / Tracking Station) prove scenarios 6 (an uncrewed RC-L01 relays without control), 11 (a looped recording whose real run is over has no node) and 12 (a playback-disabled recording relays), and that a registered ghost node is the only way home for a free endpoint (the relay half of 1, and the control-source half of 3). Also flown and armed: `CN-2-ghost-commnet-live-probe` proves 1, 3 and 8 on a real live probe's own `ControlPath` (the DD1 probe behind Duna; reading run `2026-09-26_2030`); `CN-3-ghost-commnet-timeline-warp` proves 9, 10 and the spawn hand-off of 1 across one rails warp (reading run `2026-09-26_2053`: an orbital end spawns in the same frame at its EndUT even during warp, so no node is ever held there). Scenarios 2, 4, 5, 7, 13, 14, 15 and 16 (the held-node position) are unit-tested only.

## 15.7 Passive Resource Generation During Ghost Windows

Passive background processes (science lab processing, ore drilling, ISRU conversion) generate resources on real vessels over time. When a vessel is ghosted, these processes are not running — the ghost is a Unity GameObject, not a KSP Vessel.

**This is not a gap.** All passive earnings were already captured during the original playthrough and exist as committed game actions on the timeline. The game actions system's recalculation walk processes them regardless of ghost status. Science a lab transmits (or pays out on recovery) during the recording's time span is an immutable `ScienceLabEarning` action on the timeline; data the lab is still processing travels in the spawned vessel's snapshot. Funds from contract milestones are on the timeline. Everything is on the timeline.

After the chain tip fires and the real vessel spawns, all passive processes resume normally from that point forward. A Parsek time jump (fast-forward, Real Spawn Control warp) is not production time: both jump kinds shift every converter's `lastUpdateTime` by the jump delta, loaded or not (`ConverterJumpPolicy`). Any new passive earnings are captured when the player eventually commits.

## 15.8 Ghost Visual Effects and Audio

Every effect on a ghost is rebuilt by Parsek from the part prefabs and the recording; nothing is copied from a live vessel. Following the minimal-recording principle, the recording stores threshold crossings (the `PartEvent` stream of Section 4.3), and anything derivable at playback (reentry heating, audio choice) is derived, not stored.

- **Engines** (`EngineFxBuilder`). At ghost build, each engine module's particle FX is built from the part's EFFECTS node (or its legacy top-level `fx_*` keys); `EngineIgnited` / `EngineThrottle` / `EngineShutdown` events at their recorded UTs drive emission. The recorder keys engines by `(ulong)pid << 8 | (uint)moduleIndex`, so a part may carry up to 256 engine modules.
- **RCS.** `RCSActivated` / `RCSThrottle` / `RCSStopped` carry a normalized aggregate power; the recorder tracks them in their own dictionaries (`activeRcsKeys` / `lastRcsThrottle`), so RCS and engine keys may overlap.
- **Part state.** Deployables, gear, cargo bays, lights (including blink), fairings, parachute stages, robotics, heat animations and inventory placements replay from their events; `Decoupled` and `Destroyed` hide the part's subtree.
- **Reentry** (`GhostVisualBuilder.TryBuildReentryFx`): fire particles over a combined emission mesh, an additive flame-shell overlay and part-material glow. It is built lazily, and only for a trajectory with reentry potential (`TrajectoryMath.HasReentryPotential`: any orbit segment, or a sample at 400 m/s or more). Intensity is computed each frame from the playback speed and the body's atmospheric density and Mach number (`ComputeReentryIntensity`), matched to stock aeroFX: nothing below Mach 2.5 or near the top of the atmosphere.
- **Explosions.** A recording whose terminal state is Destroyed fires one explosion at its end (`GhostPlaybackLogic.ShouldTriggerExplosion`), and an individual `Destroyed` part event explodes at that part through stock `FXMonger.Explode` with the part's `explosionPotential`.
- **Audio** (`GhostAudioPresets`). Engine loops are chosen by propellant and thrust class (light / medium / heavy at 50 kN and 300 kN) from stock clips, independent of the part's EFFECTS AUDIO config, which sound mods delete. At most 4 looped sources play per ghost, ranked by priority and distance; explosion one-shots pick a clip by explosion power.

## 15.9 Mod Compatibility for Ghost Visuals

Ghosts load part meshes from whatever parts the original vessel used, including modded parts. Base mesh rendering works for any part. Custom visual effects from mods may require explicit support. Unsupported modded visuals degrade gracefully — the mesh is in the right place but the custom animation doesn't play.

**Waterfall and ReStock: ghost FX recovery.** Waterfall config packs (Stock Waterfall Effects) replace stock engine and RCS particles with Waterfall plumes and delete the stock EFFECTS definitions the ghost FX are built from; ReStock re-authors stock parts' EFFECTS in its ModuleManager patches. Parsek recovers a ghost plume from the pristine configs instead of drawing nothing:

- `WaterfallCompat.cs` is the per-part gate (a name check for `ModuleWaterfallFX`, with no compile-time Waterfall reference). With the gate closed, a stock install behaves identically.
- `PristinePartFxResolver.cs` reads the part's pre-ModuleManager EFFECTS node, per-ordinal engine / RCS effect names and legacy `fx_*` keys from its on-disk `.cfg` (ModuleManager patches the GameDatabase in memory only); `EngineFxBuilder.TryApplyPristineEngineFxFallback` and `GhostVisualBuilder.TryApplyPristineRcsFxFallback` consume it.
- `ReStockPatchFxIndex.cs` is a lazy per-session index of the EFFECTS ReStock authors for stock parts, parsed from ReStock's patch files on disk. A missing ReStock directory is a permanently empty index. `HasAuthoredEffectsFor` stands down Parsek's hardcoded stock per-part FX tunings for a part ReStock re-authored (gated on ReStock, not on Waterfall).
- Two hard rules. Never add `Effects/` to the shared `TryResolveFxPrefabExact` probes: legacy `fx_*` names resolve through KSP's builtin `Effects/{name}` path on their own, and stock paths rely on the deliberate `fxPrefabFallbacks` substitutions. Match a fresh `EFFECTS` patch node by its exact name, never `!EFFECTS` (a delete).

Parity instruments are `GhostFxFingerprint.cs` (`[FxFingerprint]` lines per built part) and `GhostFxEmissionProbe.cs` (`[FxEmissionProbe] measured:`, the measured particle direction). The full contract is `docs/dev/mod-compatibility-notes.md`, "Waterfall ... and ReStock".

## 15.10 Watch Mode (Camera Follow)

`WatchModeController` (owned by `ParsekFlight`, flight scene only) locks the flight camera onto a ghost so the player can follow a replay, then hands the camera back. It is a camera only: the ghost stays visual and the active vessel stays under KSP's control.

- **Entry.** `EnterWatchMode(index)` is reached from the Watch buttons of the Recordings table, the Missions window and the Timeline; pressing it again on the watched recording exits. Entry is refused for a recording suppressed by an active Re-Fly session, for a recording with no ghost state, for a ghost on another body than the active vessel, and for a ghost at or beyond `WatchEnterCutoffMeters` (300 km) from the active vessel, which posts `Ghost too far to watch (Nkm, max 300km)`: the floating origin, terrain, atmosphere and skybox are all anchored to the active vessel. If the active vessel is not in a safe situation, `Your vessel continues unattended` is shown.

- **While watching.** A control lock ( `ParsekWatch` : staging, throttle, vessel switching, EVA input, camera modes, pitch) keeps keys from reaching the unattended vessel, and KSP's pivot tracking is turned off so Parsek drives the camera. `V` toggles Free / Horizon-Locked framing, `W` cycles to the next watchable ghost (not debris, on the same body, within visual range), and `[` or `]` exits (Backspace is stock's Abort key). Inside the exit cutoff the watched ghost is forced to full fidelity (mesh, part events, positioning) whatever its rendering zone.
- **Range exit.** The camera leaves a ghost that passes `WatchExitCutoffMeters` (305 km, a 5 km hysteresis band over entry) for `WatchExitCutoffDebounceFrames` (3) consecutive frames, so one stale distance across a floating-origin seam does not exit. Three consecutive frames with no usable camera target also exit ( `WatchNoTargetExitFrames` ).
- **Chains and loops.** When the watched recording ends, the camera follows onto its continuation ( `TransferWatchToNextSegment` , keeping the camera framing), during a short end hold when that ghost has not spawned yet. On a loop cycle boundary or an overlap copy rebuild the camera is bridged on a temporary anchor at the ghost's last pose and retargeted to the new cycle's ghost; a destroyed cycle holds on the explosion first. Entering watch on a looping ghost that is beyond visual range resets its loop phase so it replays from the start, unless its current phase is itself watchable.
- **Exit and restore.** `ExitWatchMode` removes the lock, restores the pivot sharpness, and returns the camera to the vessel, distance, pitch and heading saved at fresh entry (hops between ghosts keep the original); if that vessel is gone it falls back to the active vessel. If the active vessel is destroyed while watching, the camera restore is skipped and KSP assigns the new vessel. An automatic exit (range cutoff, end of playback, lost target) keeps the watched lineage protected until a computed UT ( `GhostPlaybackLogic.ComputeWatchLineageProtectionUntilUT` ), so the watched flight's debris does not vanish the moment the camera leaves.
- **Index contract.** The controller keys its state by committed-list index plus recording id and rebinds through the committed-list notifications ( `OnRecordingInserted` / `OnRecordingDeleted` ).

---

## 16. Data Flow Summary

### 16.1 Recording Phase

```
Player flies missions, switches vessels, interacts with world
  |
  +- Focus vessel -> ACTIVE recording (full trajectory + all events)
  +- Physics-bubble vessels -> BACKGROUND recording (reduced trajectory + discrete events)
  +- On-rails vessels -> ORBITAL CHECKPOINTS at key moments
  +- Focus switches -> logged with UT
  +- Environment transitions -> trigger track section boundaries
  +- Reference frame transitions -> trigger track section boundaries
  +- Scene changes / time warp -> checkpoint all session vessels

Output: RecordingTree with multi-vessel, multi-track data
```

### 16.2 Merge Phase

```
RecordingTree
  |
  +- For each vessel: select highest-fidelity track at each UT
  +- Stitch into continuous single-source tracks
  +- Snap-switch at source boundaries (logged discontinuity)
  +- Collect DAG events into recording tree

Output: Merged Recording per vessel + RecordingTree (DAG)
```

### 16.3 Playback Phase

```
Committed timeline with recordings
  |
  +- Ghost chain evaluation on scene load:
  |    +- Walk committed trees for vessel-claiming events
  |    +- Ghost claimed pre-existing vessels
  |    +- Compute chain tips and spawn suppression
  |
  +- For each recording active at current UT:
  |    +- Is ghost within Zone 1? -> full mesh + part events
  |    +- Is ghost within Zone 2? -> mesh only, orbital propagation
```

```
  |    +- Is ghost in Zone 3? -> no render, track logically
  |
  +- For looped recordings:
  |    +- Is anchor vessel loaded? -> spawn ghost at current loop phase
  |    +- Position relative to anchor's current position
  |
  +- Chain-tip spawn: when recording ends, check chain status
  |    +- Intermediate link: suppress spawn, continue ghost
  |    +- Chain tip: spawn real vessel (with PID preservation)
  |    +- Apply collision detection, terrain correction, ghost extension
  |
  +- Part events fire at recorded UT for all visible ghosts
```

---

## 17. Recording File Format

### 17.1 Version Header

Every recording carries two load-gating discriminators: `RecordingFormatVersion` (currently 1) and `RecordingSchemaGeneration` (currently 4). Any code that reads a recording or sidecar checks both first via `RecordingStore.IsRecordingSchemaCompatible`. A mismatched format version or an older/newer generation is rejected with a clear reason ( `generation-missing` , `generation-older` , `generation-newer` , `format-version-mismatch` ) rather than attempting to parse a shape it was not built for.

### 17.2 Single Current Contract - No Migration

Pre-1.0, the recording format is a single clean-slate contract with no additive evolution, no forward tolerance, and no migration paths. Each schema-generation bump is a hard reset that deletes the prior generation's tolerance seams, so a loader never sees a shape it was not built for. Recordings written by a different generation are rejected on load, not partially parsed or played back at reduced fidelity.

### 17.3 Separation of Concerns

```
Save file (ScenarioModule — persisted in .sfs):
  RecordingTree metadata:
    - Id, TreeName, RootRecordingId, ActiveRecordingId
    - BranchPoints (full serialization including type-specific metadata)
    - Resource deltas (PreTreeFunds/Science/Rep, DeltaFunds/Science/Rep)
  Per-Recording metadata:
    - RecordingId, TreeId, VesselName, VesselPersistentId
    - DAG linkage (ParentRecordingId, ParentBranchPointId, ChildBranchPointId)
    - Loop config (LoopPlayback, LoopInterval, LoopAnchorVesselId, LoopAnchorBodyName)
    - Spawn state (VesselSpawned, SpawnedVesselPersistentId)
    - Terminal state, rewind save references
    - Sidecar file path reference
  Committed tree list (ordered by commit time)
  NOT persisted: ghost chain state (re-derived on every load from committed trees)
  NOT persisted: BackgroundMap (runtime only)

Sidecar files (one .prec file per RecordingTree):
  - Format version header
  - TrajectoryPoints (per Recording, per TrackSection)
  - OrbitSegments (per TrackSection, plus legacy flat list)
  - PartEvents (per Recording)
  - SegmentEvents (per Recording)
  - TrackSections (environment, reference frame, source, boundaries)
  - VesselSnapshot and GhostVisualSnapshot (ConfigNode blobs)
  - TerrainHeightAtEnd (per Recording)
  - ControllerInfo list (per Recording)
```

This separation keeps saves lightweight and makes file-sharing natural — you share sidecar files, not save games.

---

# 18. Performance Budget

## 18.1 Cost Model

Costs that scale with ghost count:

- Unity draw calls (mesh rendering) — dominant cost for visual ghosts
- Part event processing (checking if any event fires this tick)
- Trajectory interpolation (computing position from recorded frames)

Costs independent of ghost count:

- Recording (only physics bubble matters)
- Timeline management
- Orbital propagation (cheap analytical math)
- Ghost chain evaluation (microseconds for typical tree sizes of 5-20 committed trees)

## 18.2 Profiling Guidance

The distance LOD thresholds (Section 11.4) were set from the Phase 11.5 playtest measurements and are backend-owned constants, not player settings. The architecture (zone-based rendering, looped ghost spawn thresholds, anchor-dependent lifecycle) provides the levers to retune them from new measurements.

---

# 19. Error Recovery

## 19.1 General Principle

Parsek never crashes or corrupts a save due to bad recording data. Recording failures degrade gracefully: ghosts may be missing or visually imperfect, but gameplay continues.

Ghost chain vessel conversion (Section 13.10) introduces the first modification of real game state. If ghost conversion fails, the vessel is left untouched and current behavior applies (no paradox prevention, but no data loss). The quicksave is always available as a full backup.

## 19.2 Sidecar File Errors

- **Corrupted sidecar:** Log warning, skip recording, show as "damaged" in recordings manager.
- **Missing sidecar:** Log warning, show as "missing data." Timeline metadata preserved.
- **Incompatible schema (format version or generation mismatch):** Reject on load via `RecordingStore.IsRecordingSchemaCompatible` with a clear reason (`generation-missing` / `generation-older` / `generation-newer` / `format-version-mismatch`). Do not attempt to parse - pre-1.0 there is no migration path.
- **Partially readable:** Parse the intact portion; a sidecar that cannot be parsed structurally is skipped and shown as "damaged." There is no forward-tolerance path that ignores unknown fields and plays back at reduced fidelity.

## 19.3 DAG Structure Errors

- **Orphaned segment:** Treat as root segment. Ghost plays from its own trajectory data.
- **Empty child segment:** Skip that child. Other children play normally.
- **Missing merge target:** Treat as terminal event. Ghost despawns at merge UT.
- **Circular reference:** Detect during load (topological sort fails). Reject as corrupted.

## 19.4 Trajectory and Checkpoint Errors

- **NaN/Infinity positions:** Skip affected frames. Use last valid frame.
- **Invalid Keplerian elements:** Skip ghost for the interval. Attempt recovery from next valid checkpoint.
- **Position underground:** Allow it (can be legitimate). Only flag if below body center.
- **Large discontinuity (>1km):** Log warning. Interpolate over 1 second to smooth.

### 19.5 Playback and Ghost Errors

- **Anchor vessel missing:** Mark loop as broken. Notify player: "Route to [vessel name] is broken — vessel not found."
- **Anchor on wrong body:** Mark loop as broken.
- **Docking port occupied:** Ghost plays approach but despawns before dock event. Loop continues cycling.
- **Part type missing (mod removed):** Skip visual event for that part. Log on first occurrence per part type.
- **Ghost mesh fails to instantiate:** Skip ghost, retry on next load.
- **Too many ghosts:** Distant ghosts degrade through the distance LOD (11.4); there is no count cap and no hard failure.

### 19.6 Ghost Chain Errors

| Failure | Recovery | Data Loss |
|---|---|---|
| Chain walker throws exception | Skip all ghosting. All vessels remain real. No paradox prevention. | None |
| Snapshot fails | Skip this vessel's ghosting. Vessel remains real. | None |
| Despawn fails after snapshot | Log critical error. Attempt restore from snapshot. Quicksave as backup. | Possible — vessel may need quicksave reload |
| Ghost GO creation fails after despawn | Vessel despawned but invisible. Chain walker tracks it as ghosted (spawns at tip). Player sees vessel disappear then reappear. | Visual gap only |
| Chain-tip spawn fails | Ghost persists past tip. Player reloads quicksave. | Vessel stuck as ghost until reload |
| Save during partial ghost conversion | On load, chain walker re-evaluates. Both save and quicksave would need to be corrupted for data loss. | Extremely unlikely |

**Principle:** Every failure path defaults to "leave the vessel alone" or "the quicksave has it." The worst case (despawn succeeds but everything else fails) is recoverable via quicksave reload. A clear error message is shown: "Ghost conversion failed for [vessel]. Load your quicksave to recover."

### 19.7 Persistence Errors

- **Orphaned timeline entries:** Show as "missing data" in manager.
- **Orphaned sidecar files:** Ignore on load, preserve files.
- **Sidecar versions and collection** (`docs/dev/design-load-restores-the-save.md` 4.4-4.5): each `RECORDING` node names its `.prec` / `_vessel.craft` / `_ghost.craft` by content hash (`precHash` / `vesselHash` / `ghostHash`; additive, no generation bump; `sidecarEpoch` keeps its meaning). A rewrite or delete files the replaced head under `Parsek/Recordings/_versions/<id>.<hash><suffix>`, and a cold load reads (and, when its save is the only owner, promotes) the version its save names. Only `ParsekFileCollector` removes Parsek files: a version, head, rewind point file or `parsek_rw_` rewind save goes when no save in the folder (including `Backup/`) and no in-memory state names it; heads go to a dated `_quarantine` folder, are deleted after 14 days if still unnamed, and come back to their place when a save names them again; an unreadable save stops the run.
- **Missing quicksave:** Rewind to that recording unavailable. Fall back to nearest prior quicksave.
- **Save during active recording:** Parsek captures session state for resume. If this fails, in-progress recording is lost but committed recordings are unaffected.

### 19.8 Broken Install and Failed Patches

Parsek does not warn about which other mods are installed; a mod it cannot cope with degrades with a log line. Two startup EVENTS are told to the player on screen, once per game start and never on a correct install (`StartupNotices.cs` pure, `ParsekHarmony.cs` collect and post):

- **Failed Harmony patches.** Each `[HarmonyPatch]` class maps to a player-facing feature by name prefix (`StartupNotices.PatchFeatureFor`); the notice gives the failed count and up to three affected features (for example crew or tech tree reservations). A unit test fails when a new patch class has no feature rule.
- **Install location.** The loaded `Parsek.dll` must sit in a folder directly under `GameData`. A nested `GameData`, the zip's own wrapper folder, or a loose DLL posts a notice naming where it was found and how to fix it.

Stock `ScreenMessages` drops a post made during a scene load, so `StartupNoticePoster` waits for a playable scene to stay current for 3 s, posts through `ParsekLog.TryScreenMessage`, and re-queues a dropped notice for the next frame. Contract: `docs/dev/mod-compatibility-notes.md`, "Startup notices".

## 20. Schema Compatibility

### 20.1 Single Current Generation - No Migration

Pre-1.0, Parsek does not carry compatibility with older recordings. The recording schema is a single clean-slate contract: `RecordingFormatVersion = 1` plus a `RecordingSchemaGeneration` discriminator (currently 4). Loads reject any recording or sidecar whose format version differs, or whose generation is missing / older / newer, via `RecordingStore.IsRecordingSchemaCompatible`. There is no migration, no additive back-compat, and no retroactive re-interpretation of old payloads - each generation bump deletes the tolerance seams that only existed to read the prior generation (for example, generation 3 retired the last pre-reset seams, including the legacy v5 world-offset RELATIVE contract; generation 4 renamed the parent-anchor ConfigNode key). A save carrying recordings from a superseded generation is rejected on load rather than migrated.

### 20.2 Ghost Chain State Is Never Persisted

Ghost chain state is not persisted - it is re-derived from committed recordings on every load. This guarantees deterministic behavior: the same committed recordings plus the current UT always produce the same chains.

### 20.3 Ghost Conversion of Quicksave Vessels

On rewind, the quicksave loads vessels that existed at recording start. Claimed vessels are despawned from the quicksave state, with their crew taken off first and set Available (`ClaimedVesselRemoval`): stock `Vessel.Die()` on an unloaded vessel kills everyone aboard, and a claimed vessel is held back, not destroyed. The chain tip's spawn seats the crew its snapshot carries. The quicksave itself is never modified - it remains a full backup. Loading the quicksave directly (bypassing Parsek rewind) restores all vessels with no ghosting.

---

## 21. Implementation Status

### 21.1 Completed Phases (1-5)

The recording system core is fully implemented:

- **Phase 1:** Segment boundary rule, SegmentEvents, crash coalescing, environment taxonomy, reference frames, orbital checkpoints, background part events.
- **Phase 2:** Multi-vessel sessions, focus switching, background trajectory recording with proximity-based sample rate, highest-fidelity-wins merge.
- **Phase 3:** Relative-frame recording, anchor-relative loop playback, loop phase tracking across vessel load/unload.
- **Phase 4:** Rewind procedure, spawn-at-recording-end, warp spawn queue, PID-based deduplication, timeline immutability.
- **Phase 5:** Distance-based rendering zones, ghost soft caps with priority-based despawning (the soft caps were replaced by the distance LOD in 0.8.1, see 11.4).

### 21.2 Phase 6: Vessel Interaction Paradox Extension — COMPLETE

The ghost chain system, spawn safety, time jump, and ghost world presence are implemented, covered by unit tests and the in-game test framework.

| Phase | Scope | Status |
|---|---|---|
| 6a — Ghost Chain Infrastructure | Chain walker, claiming logic, intermediate spawn suppression, PID preservation | Done (105 tests) |
| 6b — Ghost Conversion + Chain-Aware Spawn | Real-to-ghost conversion, chain-aware spawn-at-end, save/load re-evaluation | Done (79 tests) |
| 6c — Spawn Safety | Bounding box collision, ghost extension, terrain correction, trajectory walkback | Done (93 tests) |
| 6d — UI | Spawn warnings, ghost labels, chain status display | Done (27 tests) |
| 6e — Relative-State Time Jump | Discrete UT skip, TIME_JUMP event | Done (27 tests) |

| Phase | Scope | Status |
|---|---|---|
| 6f — Ghost World Presence | Map view, tracking station, CommNet relay (stock API), antenna specs | Done (47 tests); CommNet relay: see 15.6 |

New source files: GhostingTriggerClassifier, GhostChain, GhostChainWalker, VesselGhoster, SpawnCollisionDetector, GhostExtender, TerrainCorrector, SpawnWarningUI, TimeJumpManager, GhostMapPresence, and (rebuilt 2026-09-26 per section 15.6) GhostCommNet + GhostCommNetManager in place of the never-wired GhostCommNetRelay / AntennaSpec. The recording schema is a single clean-slate contract ( `RecordingFormatVersion = 1` , `RecordingSchemaGeneration = 4` ), not an additive versioned format.

### 21.3 Deferred Items

| Item | Reason |
|---|---|
| Full ghost visual treatment (transparency, outlines) | Ghost labels provide minimum viable distinction. Full treatment is polish. |
| Resource transfer tracking | Moot in stock KSP (requires docking, already a MERGE trigger). |
| Chain walker caching | Trivial cost for typical tree sizes. |
| Manual placement UI for total trajectory overlap | Dropped (operator ruling 2026-09-23): an exhausted walkback abandons the spawn (Section 13.7). |
| Quicksave pruning | Storage management only, no architectural impact. |

### 21.4 Since-Shipped Subsystems

Beyond the Phase 1-6 core above, later cycles shipped substantial subsystems not reflected in the original phase list:

- **Missions and Logistics:** the mission abstraction (Missions tab, mission-tree fork) and the logistics route system (milestones M1-M6, including cross-tree docking and the claw producer).
- **Re-aim solver stack:** Lambert-based re-aim / retargeting for transfers, arrival, and descent.
- **Map / tracking-station render rewrite:** the Director-driven ghost polyline and orbit-line render pipeline with the render tracer ( `MapRenderTrace` , `MapRenderProbe` ).
- **Ledger game-actions system:** fully implemented ( `GameActions/` , `LedgerOrchestrator` , nine resource modules, `KspStatePatcher` ) with the ground-truth verification harness and `LedgerTrace` observability.
- **Rewind-to-Separation (v0.9):** re-fly of committed subtrees with append-only supersede (Section 14.9).

---

*Consolidated from: recording-system-design.md, parsek-vessel-interaction-paradox.md, parsek-vessel-interaction-paradox-addendum-time-jump.md, and recording-system-extension-plan.md.*